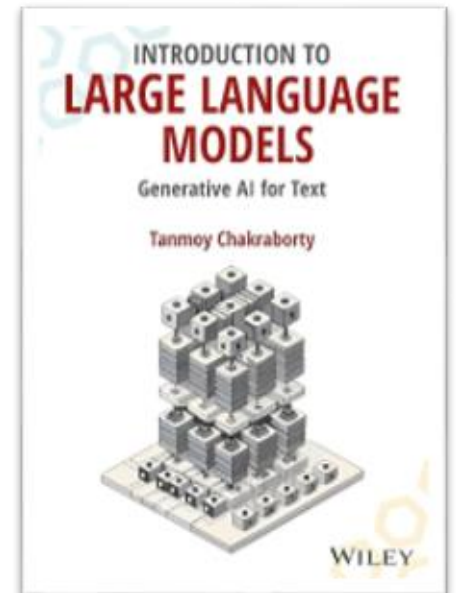


Neural Language Models



Tanmoy Chakraborty
Associate Professor, IIT Delhi
<https://tanmoychak.com/>



PC

unigram

(LMI)

Accuracy

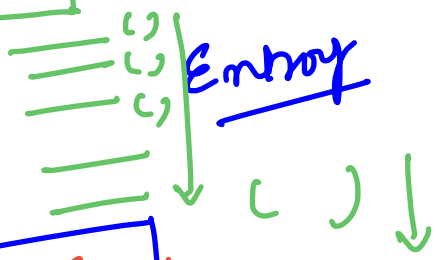
$$P(w_1, w_2, \dots, w_n) = P(w_1)P(w_2) \dots P(w_n)$$

bigram

LM2

$$P(w_1) \times P(w_2|w_1) \times P(w_3|w_2) \dots$$

$$P(w_1, w_2, \dots, w_n) = 1$$



Task-based Evaluation / Extrinsic Eval

(LMI)

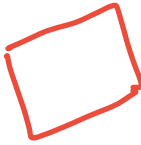
Tagore's Corp

Task

1) Fill in the blanks.

LM2

The cat sat on the _____!



Spelling Correct



NT

Intrinsic Eval

Task-agnostic Eval

Perplexity (PP)

$$= P(w_1, w_2, \dots, w_n)^{-1/n}$$

$$= \frac{1}{P(w_1, w_2, \dots, w_n)^{1/n}}$$

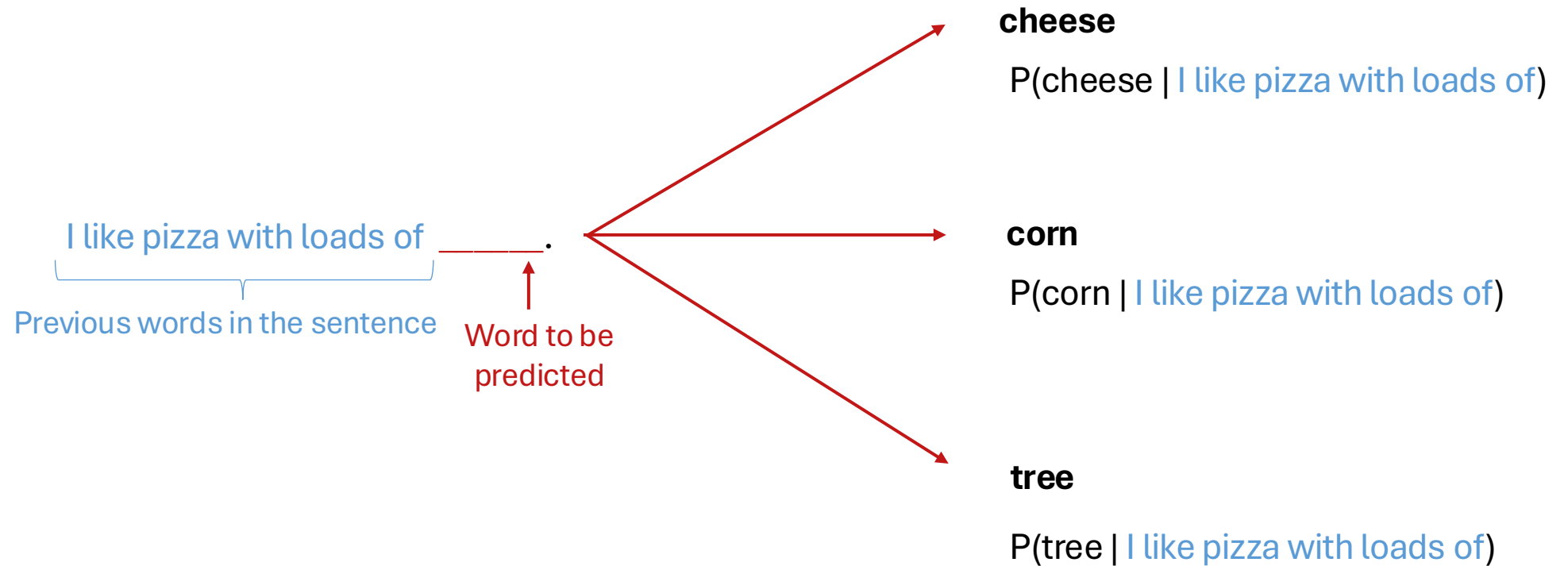
$$= \frac{1}{\sqrt[n]{P(w_1, w_2, \dots, w_n)}}$$

$$LM(1) = \frac{1}{\sqrt[n]{P(w_1)P(w_2) \dots P(w_n)}}$$

$$LM(2) = \frac{1}{\sqrt[n]{P(w_2)P(w_3|w_2) \dots}}$$

Language Modeling

- **Language Modeling** is the task of predicting what word comes next



Language Modeling

- You can also think of a Language Model as a system that assigns a probability to a piece of text.
- For example, if we have some text $x^{(1)}, \dots, x^{(T)}$, then the probability of this text (according to the Language Model) is:

$$\begin{aligned} P(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}) &= P(\mathbf{x}^{(1)}) \times P(\mathbf{x}^{(2)} | \mathbf{x}^{(1)}) \times \dots \times P(\mathbf{x}^{(T)} | \mathbf{x}^{(T-1)}, \dots, \mathbf{x}^{(1)}) \\ &= \prod_{t=1}^T \underbrace{P(\mathbf{x}^{(t)} | \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(1)})} \end{aligned}$$

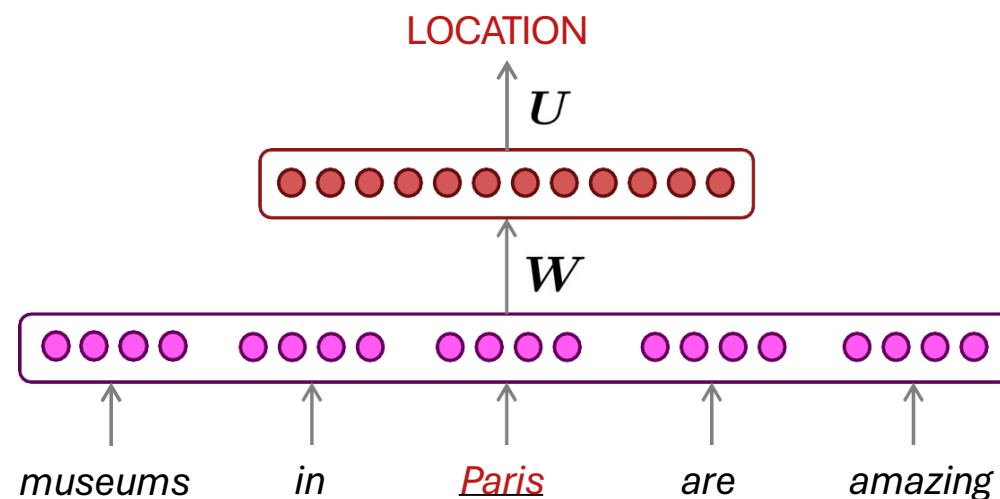
This is what our LM provides



How to Build a *Neural* Language Model?

- Recall the Language Modeling task:
 - **Input:** sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$
 - **Output:** probability distribution of the next word $P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)})$
- How about a window-based neural model?

Example: NER Task



A Fixed-window Neural Language Model

~~as the proctor started the clock~~
discard

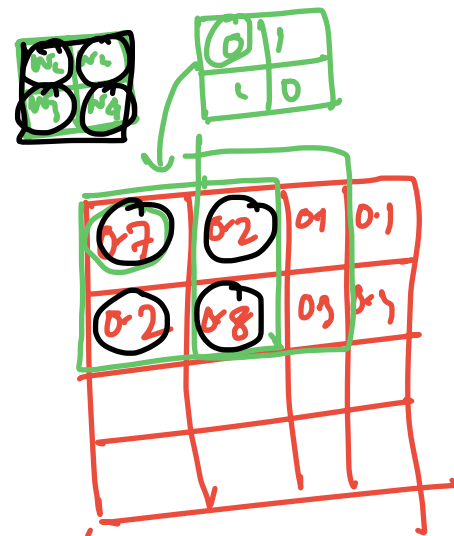
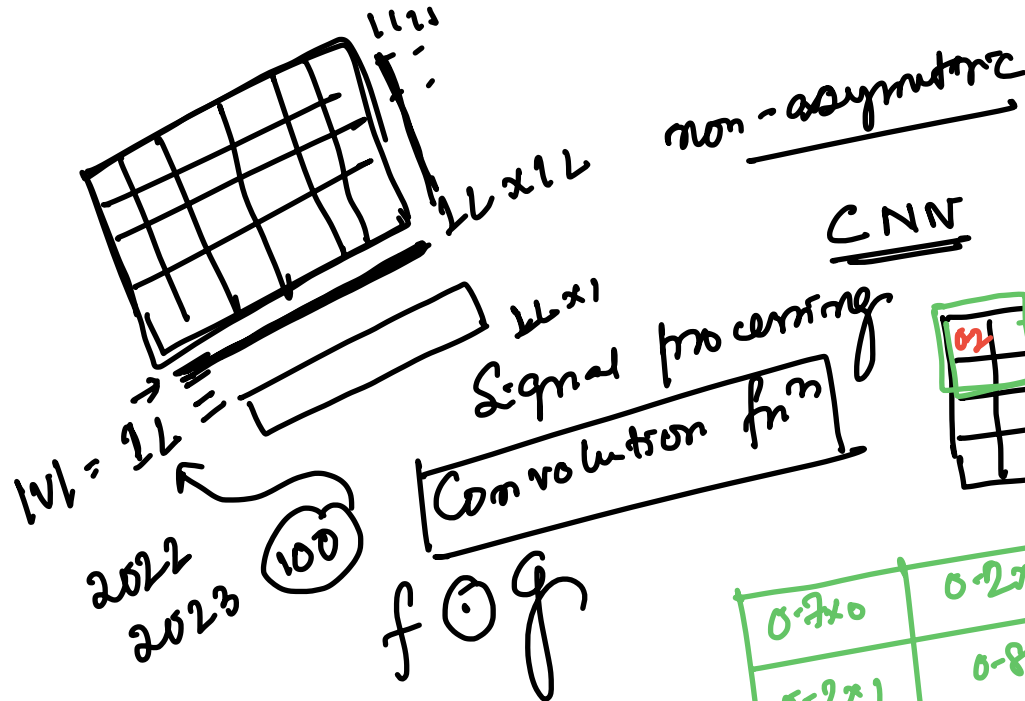
the students opened their _____
fixed window



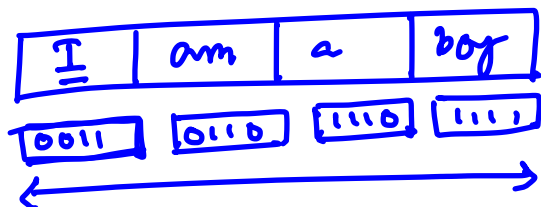
Problem of SLM

1) Sparsity

2)

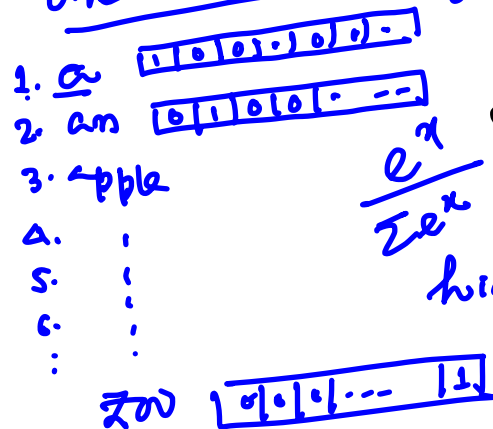


I am a boy



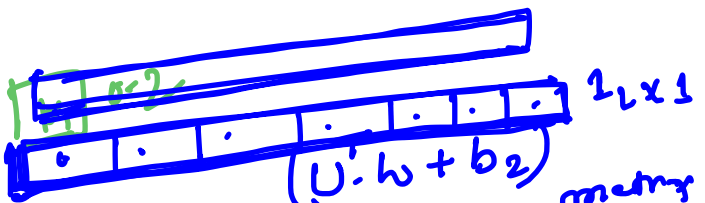
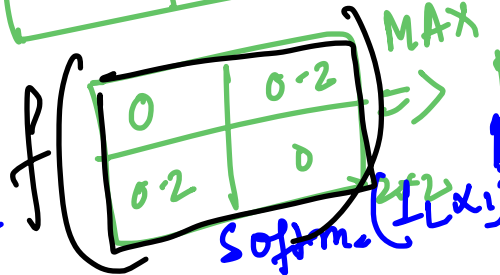
Embedding of tokens

one-hot encoding



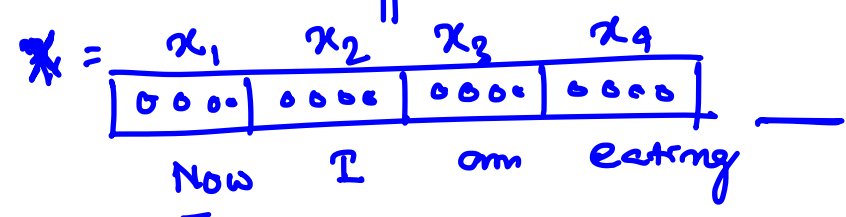
$$\frac{e^x}{\sum e^x}$$

hidden state

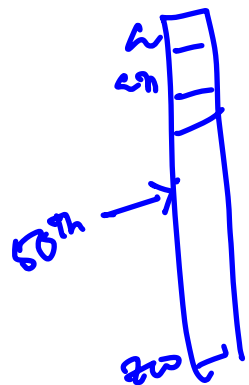


$U = \text{Trainable matrix}$
 $10 \times L = \text{unembedding matrix}$

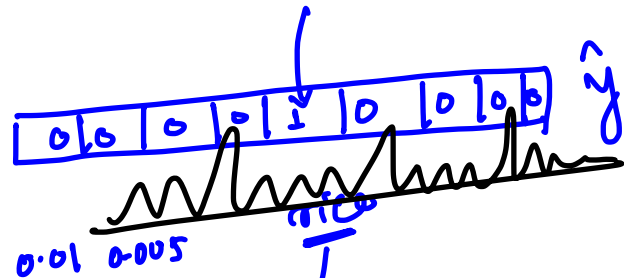
$$h_f = \frac{1}{10} (W^T \cdot X + b)$$



Now I am eating

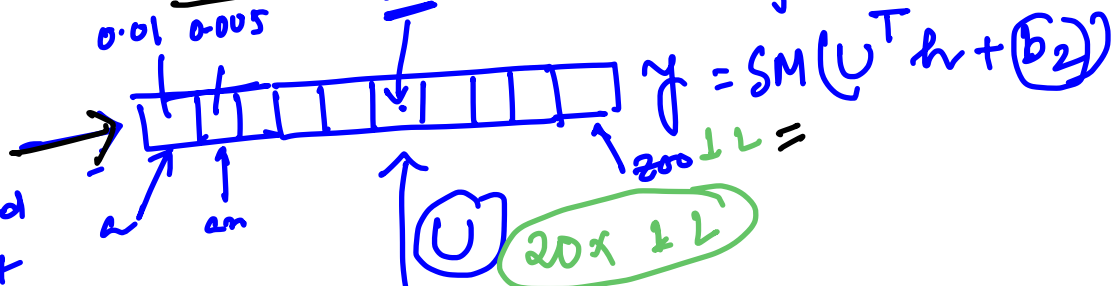


Ground truth prob dist



CE
Cross entropy ($y, \hat{y}$)

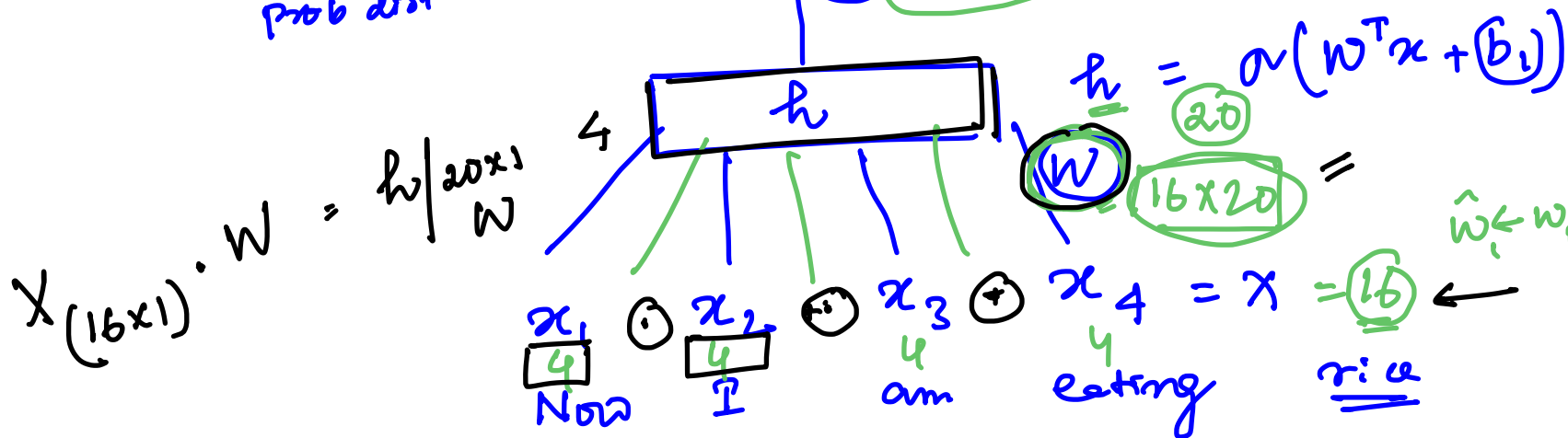
Predicted prob dist



$$y = (y_1, y_2, \dots, y_m)$$

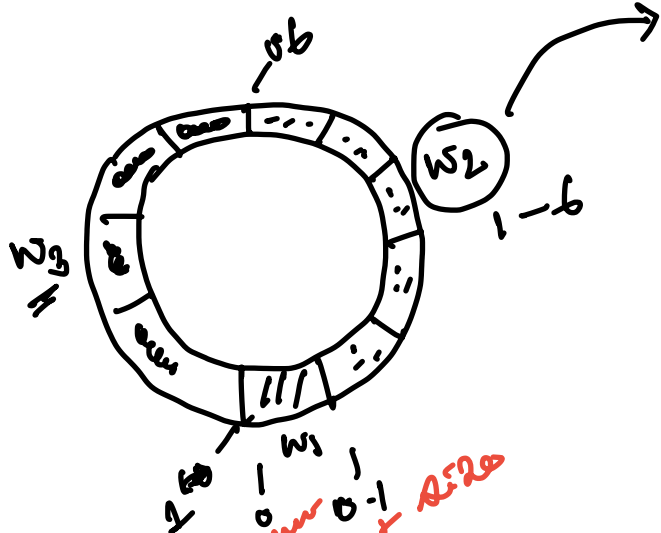
$$\hat{y} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_m)$$

$$CE(y, \hat{y}) = -y_1 \log \hat{y}_1 - y_2 \log \hat{y}_2 - y_3 \log \hat{y}_3 - \dots$$

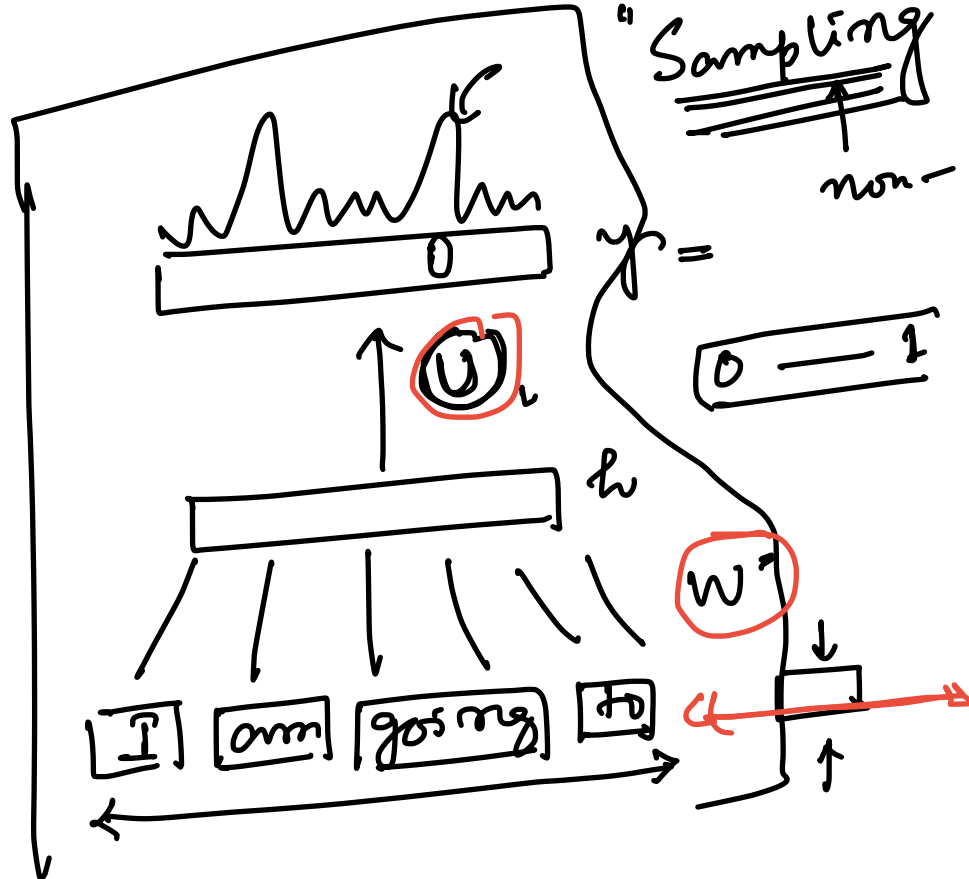


$$\hat{w}_i \leftarrow w_i - \eta \left[\frac{\partial CE}{\partial w_i} \right]$$

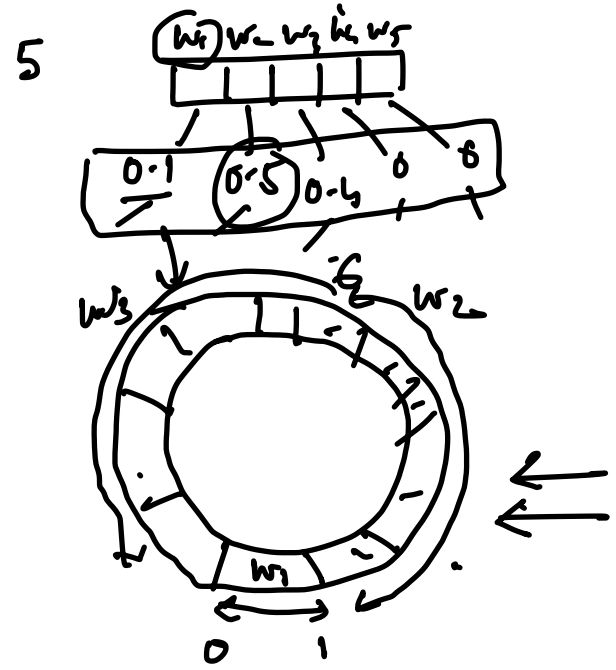
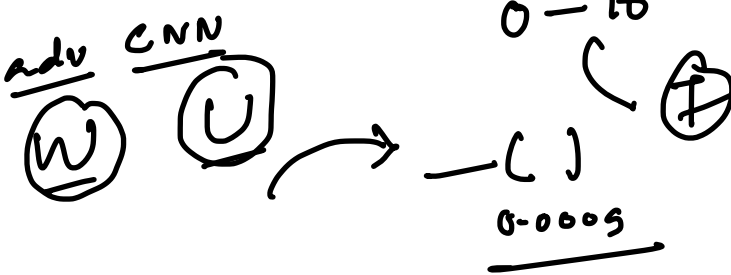
- ① Sparsity prob
- ② no need to store bigram matrix W, U

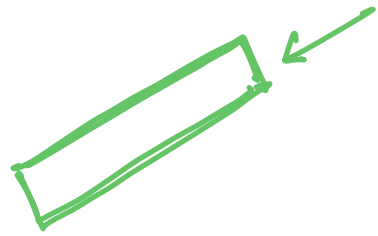


- ① new to determine size
- ② dim of W and U will change if context length and vocab will change



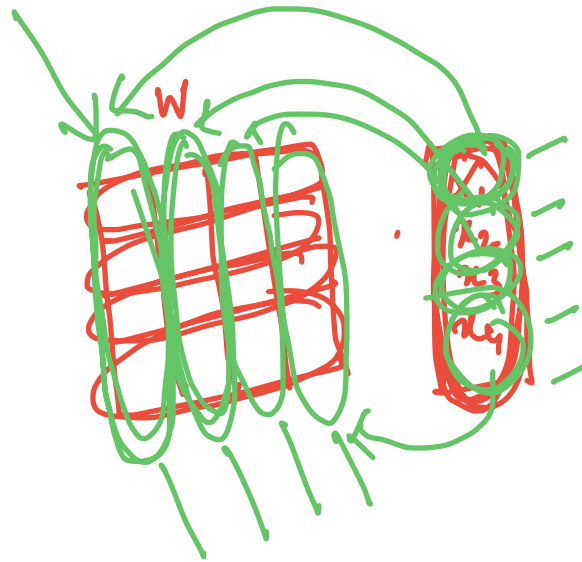
SLM





$$\sum_{i=1}^T x_i$$

RNN



- ✓ ① Content length ()
- ✓ ② w, v —
- ✓ ③ lack of memory —
padding

Now I am eating rice. I do not like it at all.

A Fixed-window Neural Language Model

output distribution

$$\hat{y} = \text{softmax}(U\mathbf{h} + \mathbf{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

$$\mathbf{h} = f(W\mathbf{e} + \mathbf{b}_1)$$

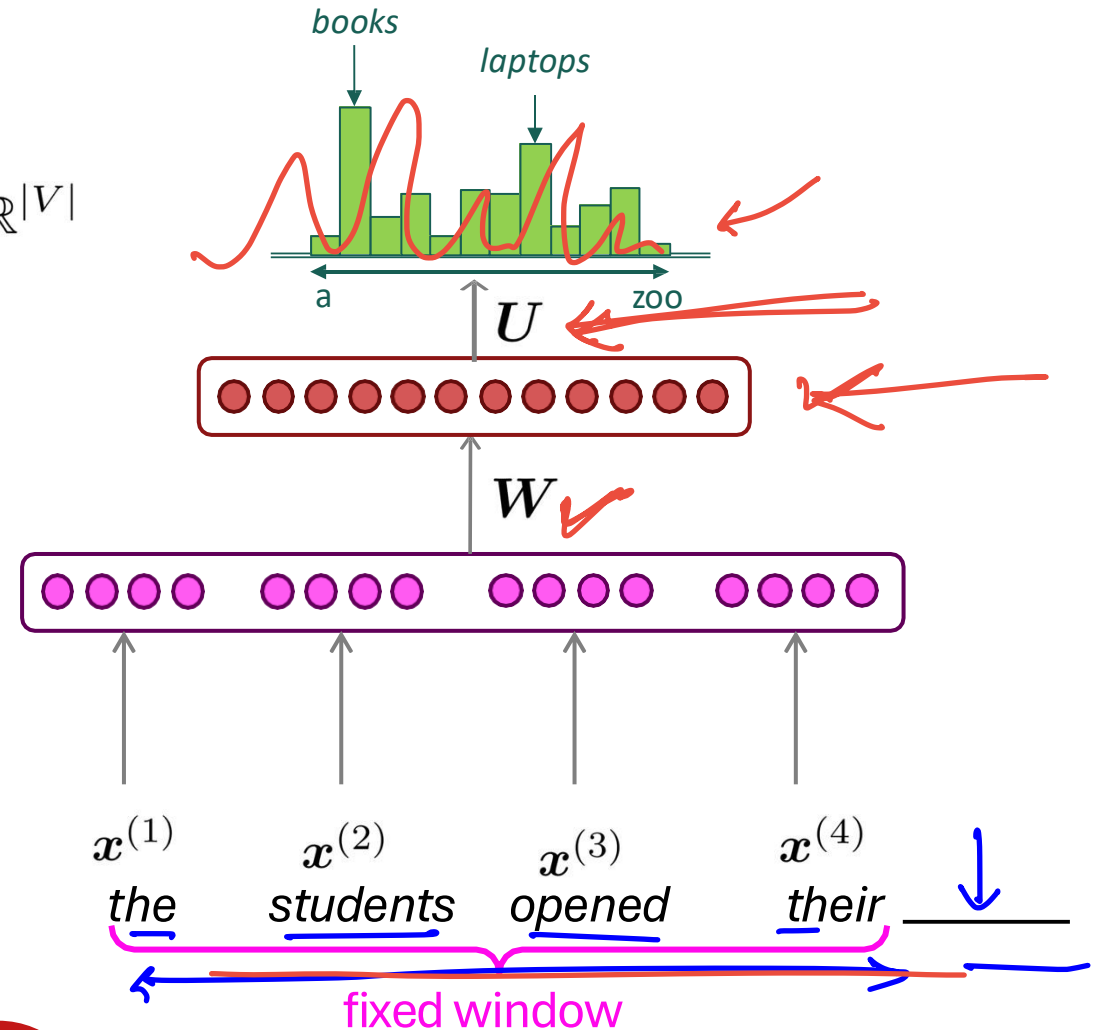
concatenated word embeddings

$$\mathbf{e} = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

words / one-hot vectors

$$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}, \mathbf{x}^{(4)}$$

as the proctor started the clock
← discard →





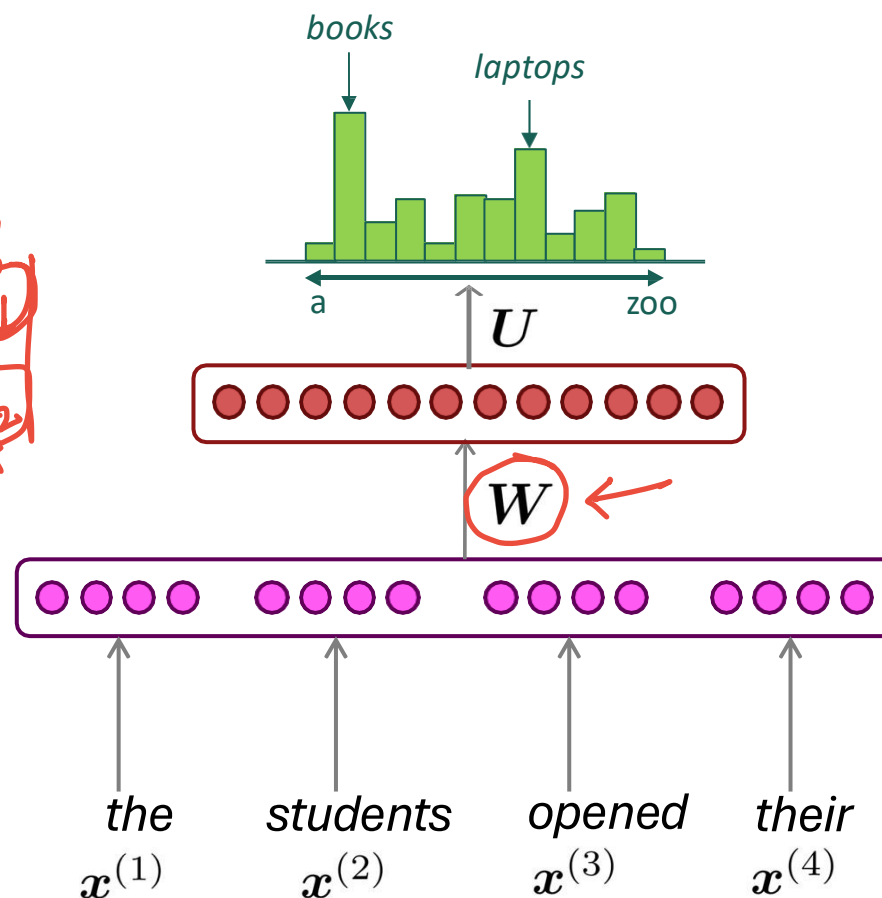
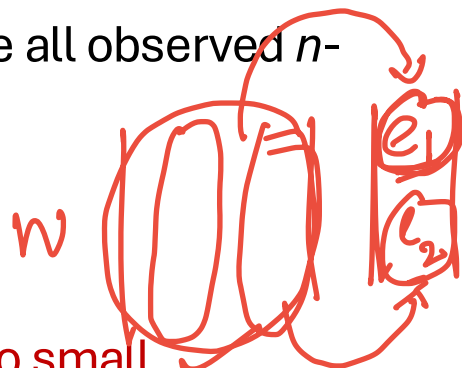
A Fixed-window Neural Language Model

Improvements over n -gram LM:

- No sparsity problem
- Don't need to store all observed n -grams

Remaining problems:

- Fixed window is **too small**
- Enlarging window enlarges W
- $x^{(1)}$ and $x^{(2)}$ are multiplied by completely different weights in W .
No symmetry in how the inputs are processed.

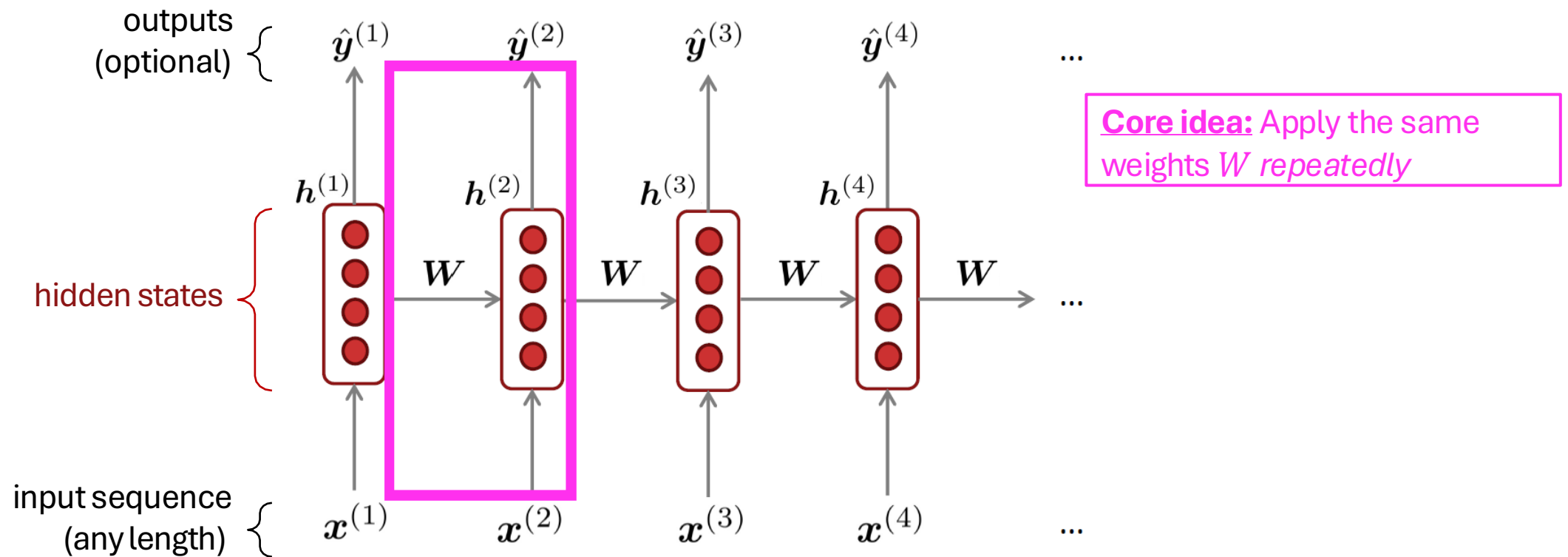


Approximately: Y. Bengio, et al.
(2000/2003): A Neural Probabilistic
Language Model

We need a neural
architecture that can
process **any length**
input



Recurrent Neural Networks (RNN)



A Simple RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax} \left(U h^{(t)} + b_2 \right) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma \left(W_h h^{(t-1)} + W_e e^{(t)} + b_1 \right)$$

$h^{(0)}$ is the initial hidden state

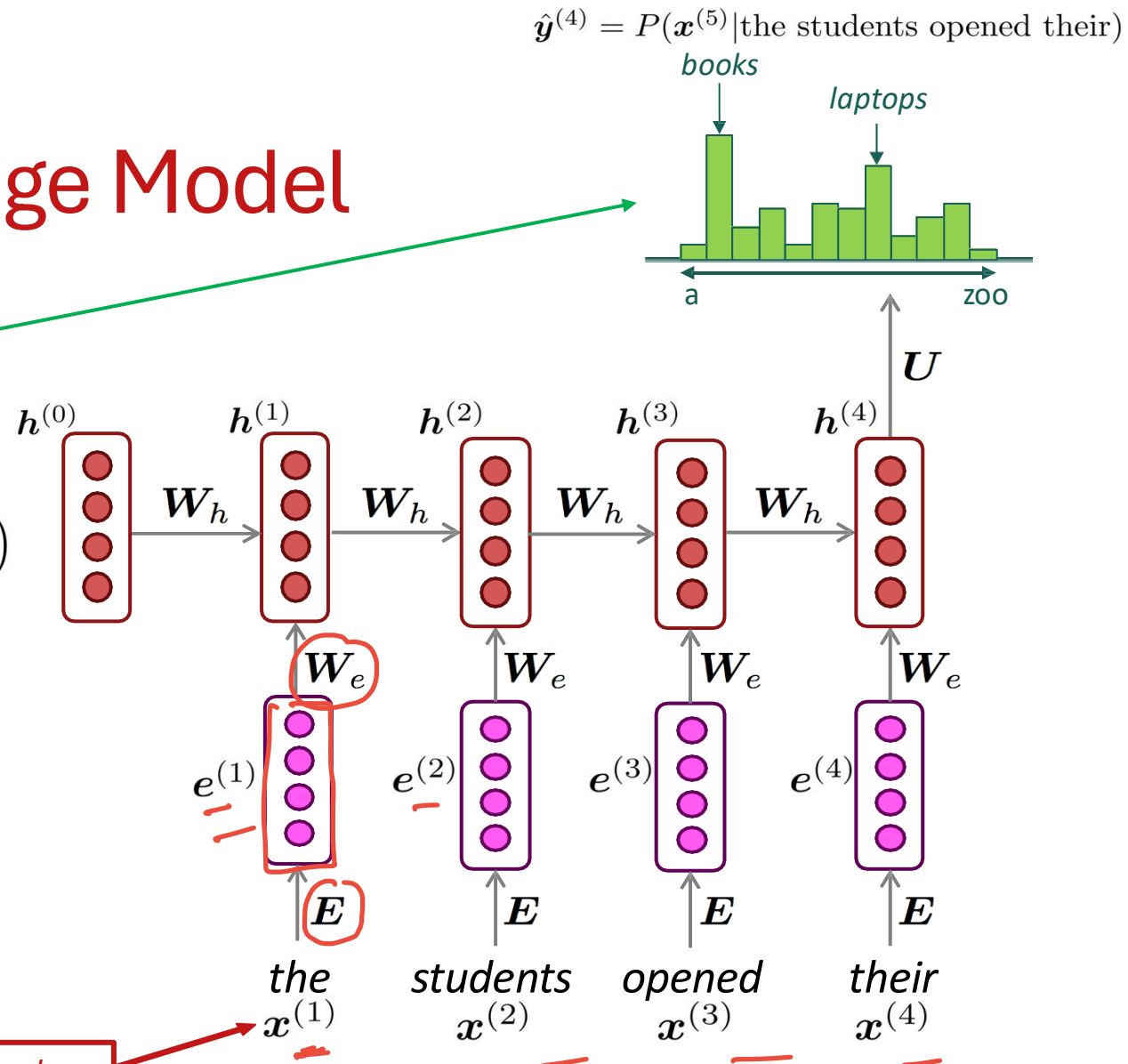
word embeddings

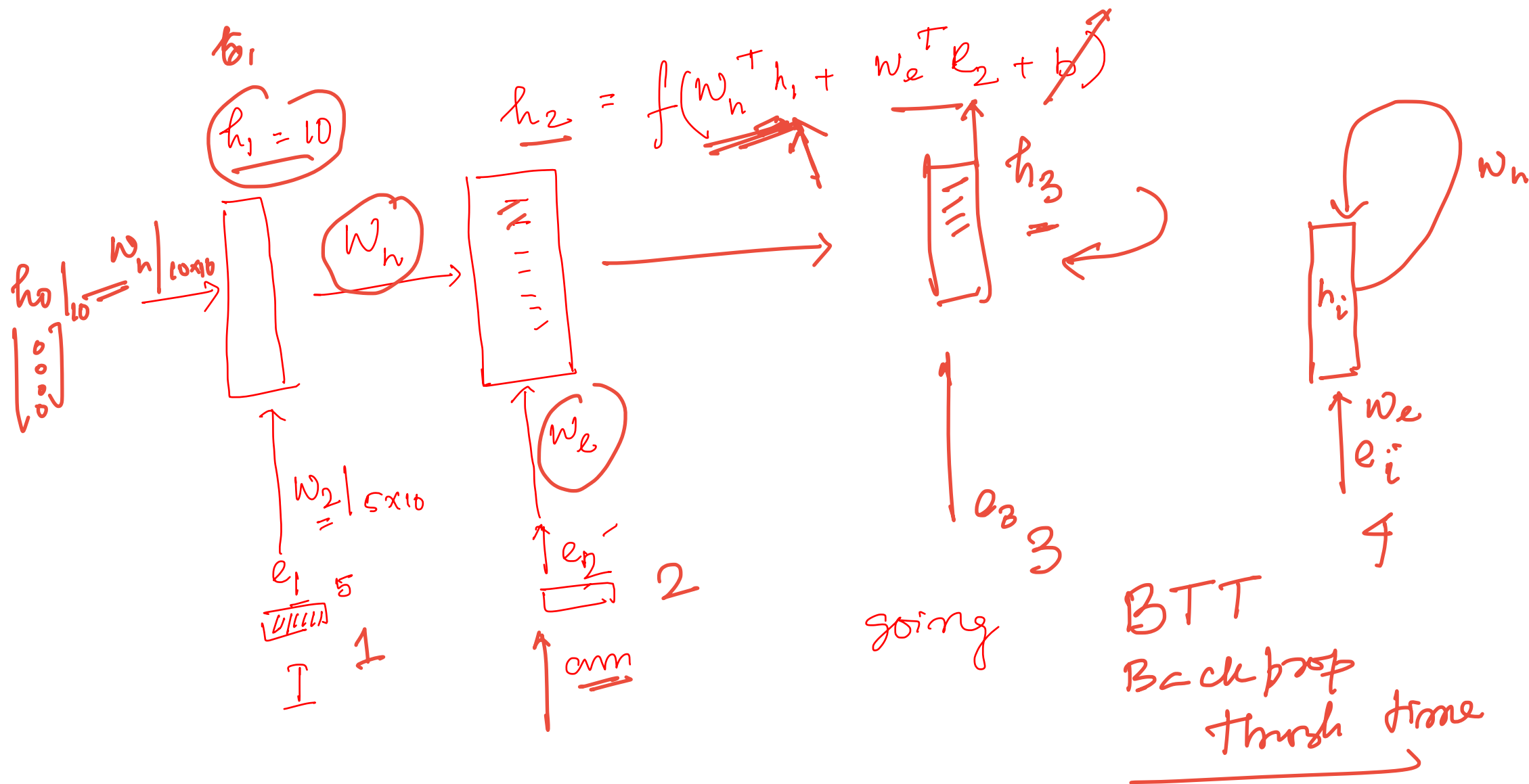
$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$

Note: this input sequence could be much longer now!





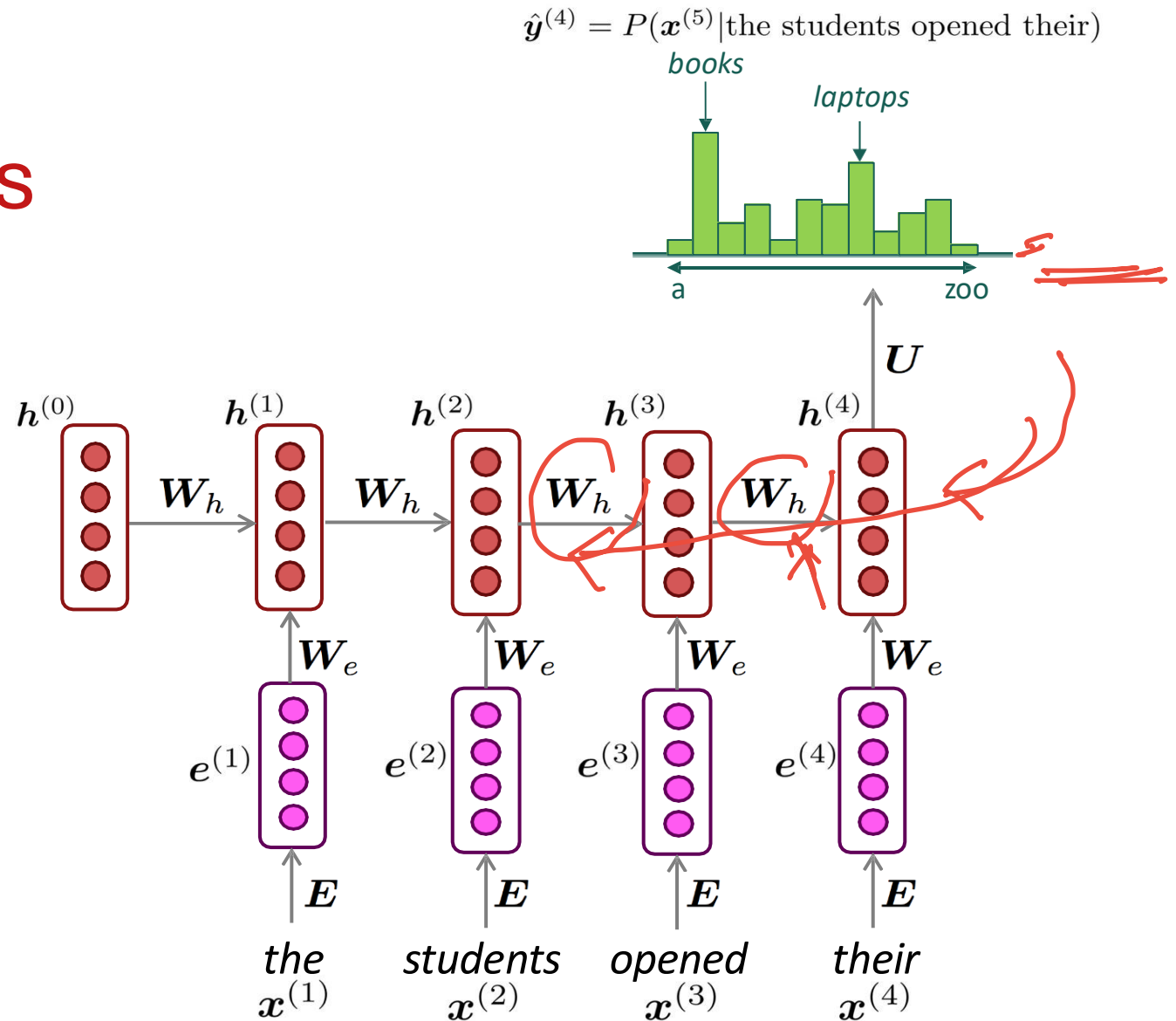
RNN Language Models

RNN Advantages:

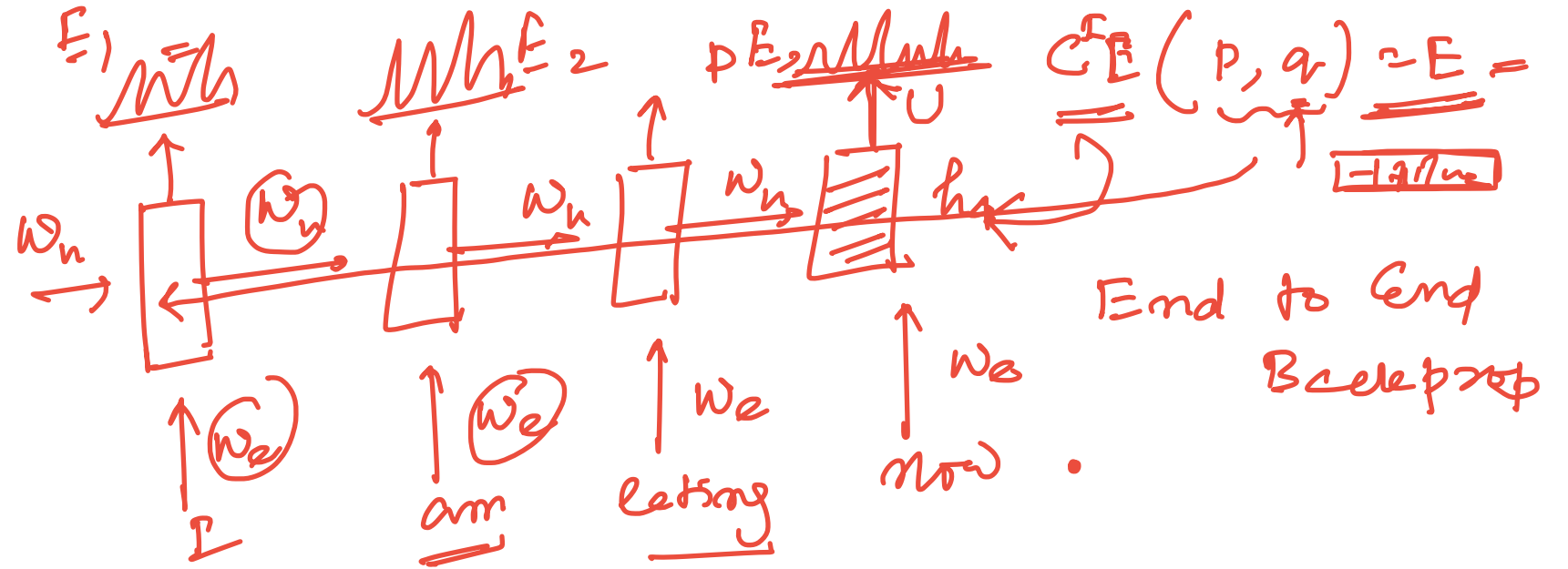
- Can process **any length** input
- Computation for step t can (in theory) use information from **many steps back**
- **Model size doesn't increase** for longer input context
- Same weights applied on every timestep, so there is **symmetry** in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is **slow**
- In practice, difficult to access information from **many steps back**



BTT



Training an RNN Language Model

Training an RNN Language Model

- Get a **big corpus of text** which is a sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{y}^{(t)}$ **for every step t** .
 - i.e., predict probability distribution of every word, given words so far
- **Loss function** on step t is **cross-entropy** between predicted probability distribution $\hat{y}^{(t)}$, and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$):

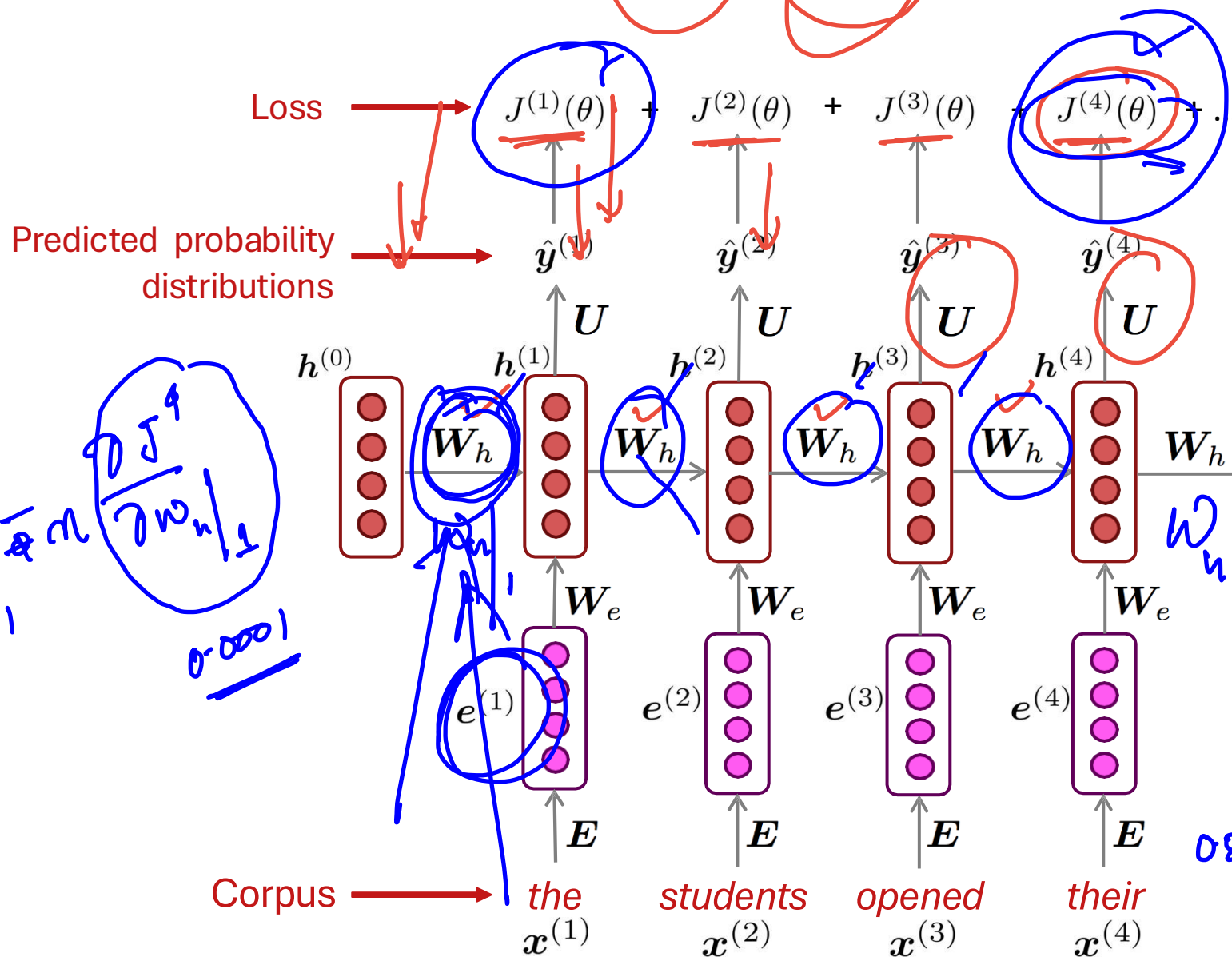
$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)} = - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$

- Average this to get **overall loss** for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$



“Teacher forcing”



$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

$$\frac{\partial J^4}{\partial w_{h/1}}$$

$$\frac{\partial J^4}{\partial w_{h/1}} =$$

$$\frac{\partial J^4}{\partial w_{h/1}} \times \dots \times \frac{\partial J^4}{\partial w_{h/1}}$$

0.0001

0.0001

0.0001

0.0001

Training a RNN Language Model

- However: Computing loss and gradients across **entire corpus** $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ at once is **too expensive** (memory-wise)!

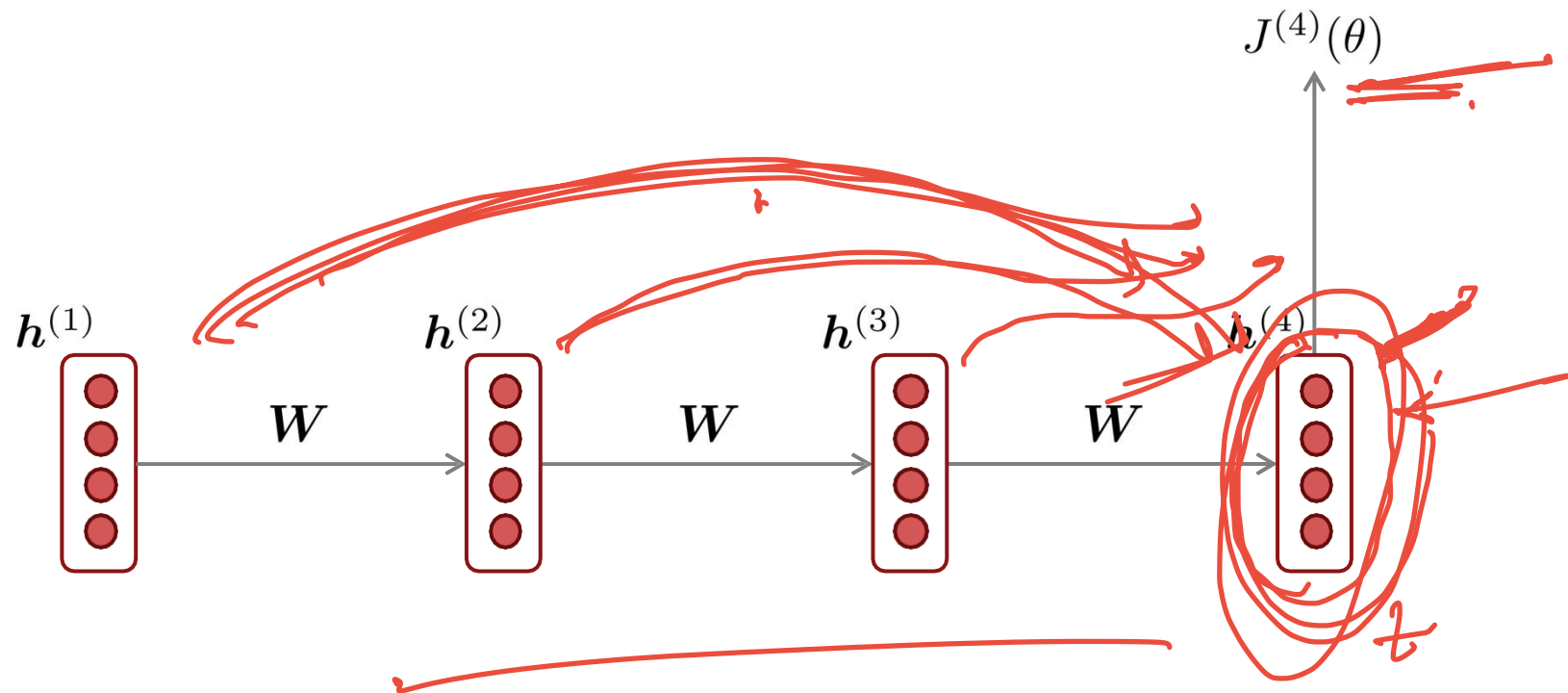
$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

- In practice, consider $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ as a **sentence** (or a **document**)
- Recall: **Stochastic Gradient Descent** allows us to compute loss and gradients for small chunk of data, and update.
- Compute loss $J(\theta)$ for a sentence (actually, a batch of sentences), compute gradients and update weights. Repeat on a new batch of sentences.



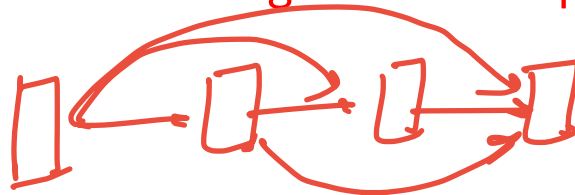
Problems with RNNs

Vanishing and Exploding Gradients



Effect of Vanishing Gradient on RNN-LM

- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____*
- To learn from this training example, the RNN-LM needs to **model the dependency** between “*tickets*” on the 7th step and the target word “*tickets*” at the end.
- But if the gradient is small, the model **can’t learn this dependency**
 - So, the model is **unable to predict similar long-distance dependencies** at test time



How to Fix the Vanishing Gradient Problem?

- The main problem is that *it's too difficult for the RNN to learn to preserve information over many timesteps.*
- In a vanilla RNN, the hidden state is constantly being rewritten

$$\mathbf{h}^{(t)} = \sigma \left(\mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_x \mathbf{x}^{(t)} + \mathbf{b} \right)$$

- How about an RNN with separate **memory** which is added to?
 - LSTMs
- And then: Creating more direct and linear pass-through connections in model
 - Attention, residual connections, etc.



Adv

- NO concept of window
- "theoretically" it captures info length content
- the size of W_x and W_h will not change if we remove the content
- it captures the sequential nature of the input
- will long range dependency

Gated mech
- LSTM/GRU

LSTMs & GRUs

- Attention

dis adv

1 Varying order

$$h_t \leftarrow h_{t-1}$$

- non-parallel

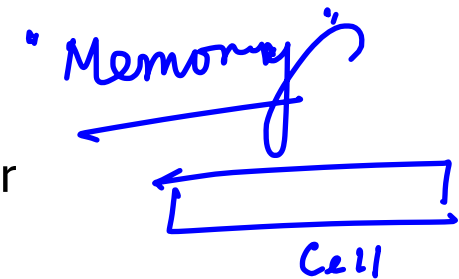
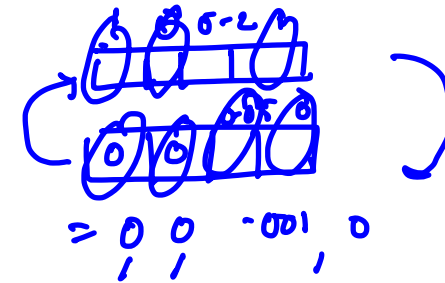
3 BN prob

1 1 1 1 (1) 1 1

Transformer

Long Short-Term Memory (LSTM)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.
- On step t , there is a **hidden state** $h^{(t)}$ and a **cell state** $c^{(t)}$
 - Both are vectors length n
 - The **cell** stores **long-term information**
 - The LSTM can erase, write and read information from the cell
- The selection of which information is erased/written/read is controlled by three corresponding **gates** (gates are calculated things whose values are probabilities)
 - The gates are also vectors length n
 - On each timestep, each element of the gates can be **open** (1), **closed** (0), or somewhere in-between.
 - The gates are **dynamic**: their value is computed based on the current context



LSTM

hidden / cell state

We have a sequence of inputs $x^{(t)}$, and we will compute a sequence of hidden states $h^{(t)}$ and cell states $c^{(t)}$. On timestep t :

Sigmoid function: all gate values are between 0 and 1

Forget gate: controls what is kept vs forgotten, from previous cell state

Input gate: controls what parts of the new cell content are written to cell

Output gate: controls what parts of cell are output to hidden state

$$f^{(t)} = \sigma(W_f h^{(t-1)} + U_f x^{(t)} + b_f)$$

$$i^{(t)} = \sigma(W_i h^{(t-1)} + U_i x^{(t)} + b_i)$$

$$o^{(t)} = \sigma(W_o h^{(t-1)} + U_o x^{(t)} + b_o)$$

New cell content: this is the new content to be written to the cell

Cell state: erase (“forget”) some content from last cell state, and write (“input”) some new cell content

Hidden state: read (“output”) some content from the cell

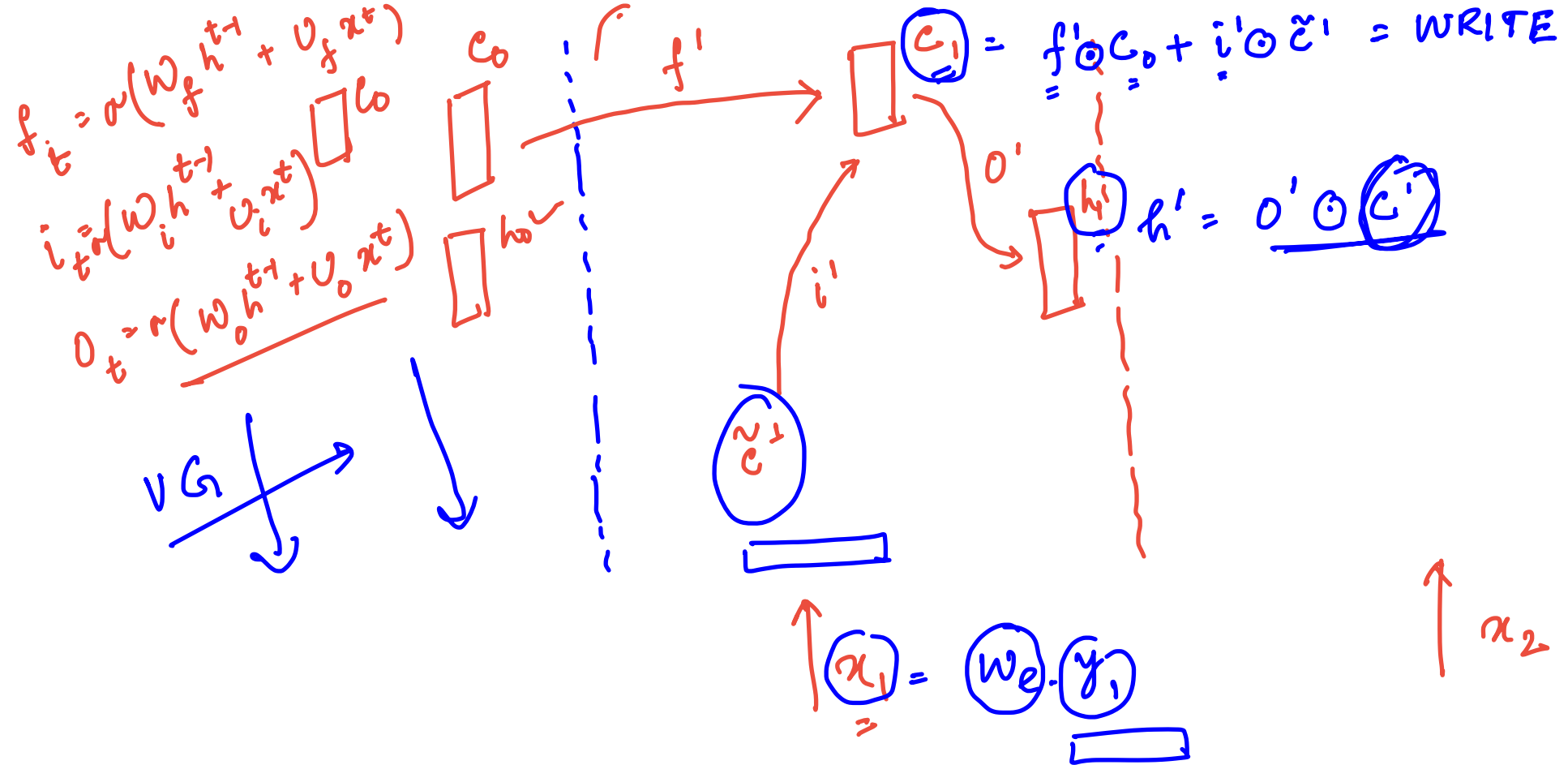
$$\tilde{c}^{(t)} = \tanh(W_c h^{(t-1)} + U_c x^{(t)} + b_c)$$

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$$

$$h^{(t)} = o^{(t)} \odot \tanh c^{(t)}$$

All these are vectors of same length n

Gates are applied using element-wise (or Hadamard) product: $\odot$



Gated Recurrent Units (GRUs)

- Proposed by Cho et al. in 2014 as a **simpler alternative to the LSTM**.
- On each timestep t , we have input $x^{(t)}$ and hidden state $h^{(t)}$ (**no cell state**).

Update gate: controls what parts of hidden state are updated vs preserved

Reset gate: controls what parts of previous hidden state are used to compute new content

New hidden state content: reset gate selects useful parts of prev hidden state. Use this and current input to compute new hidden content.

Hidden state: update gate simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

How does this solve vanishing gradient?

Like LSTM, GRU makes it easier to retain info long-term (e.g. by setting update gate to 0)

$$u^{(t)} = \sigma \left(W_u h^{(t-1)} + U_u x^{(t)} + b_u \right)$$

$$r^{(t)} = \sigma \left(W_r h^{(t-1)} + U_r x^{(t)} + b_r \right)$$

$$\tilde{h}^{(t)} = \tanh \left(W_h (r^{(t)} \circ h^{(t-1)}) + U_h x^{(t)} + b_h \right)$$

$$h^{(t)} = (1 - u^{(t)}) \circ h^{(t-1)} + u^{(t)} \circ \tilde{h}^{(t)}$$

"Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation", Cho et al. 2014, <https://arxiv.org/pdf/1406.1078v3.pdf>



LSTM vs GRU

- Researchers have proposed many gated RNN variants, but LSTM and GRU are the most widely-used.
- The biggest difference is that GRU is quicker to compute and has fewer parameters.
- There is no conclusive evidence that one consistently performs better than the other.
- LSTM is a good default choice (especially if your data has particularly long dependencies, or you have lots of training data).
- **Rule of thumb:** start with LSTM, but switch to GRU if you want something more efficient.



Sequence-to-Sequence Modeling & Attention

Neural Machine Translation?

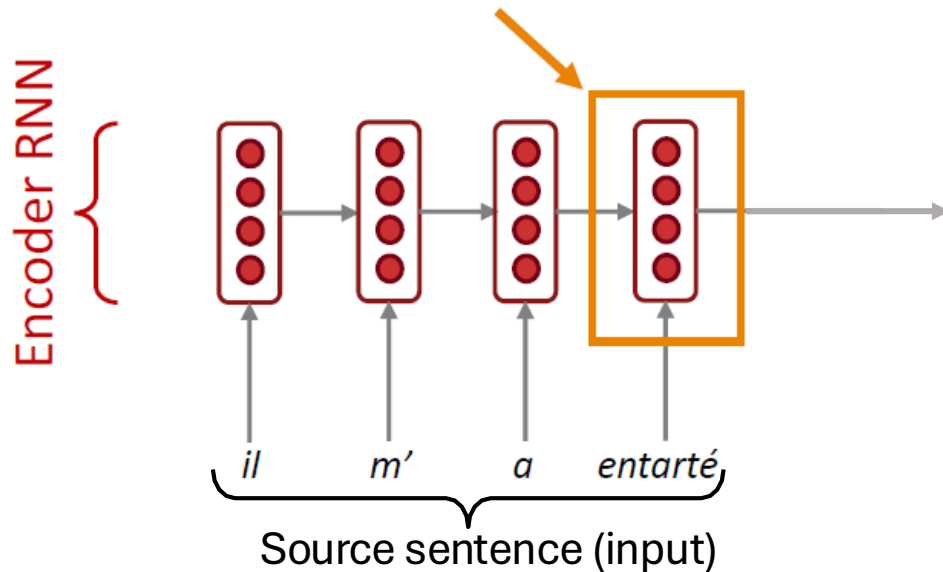
- **Neural Machine Translation (NMT)** is a way to do Machine Translation with a *single neural network*.
- The neural network architecture is called **sequence-to-sequence** (aka **seq2seq**) and it involves **two RNNs**.



Neural Machine Translation (NMT)

The Sequence-to-Sequence Model

Encoding of the source sentence.
Provides initial hidden state
for Decoder RNN.



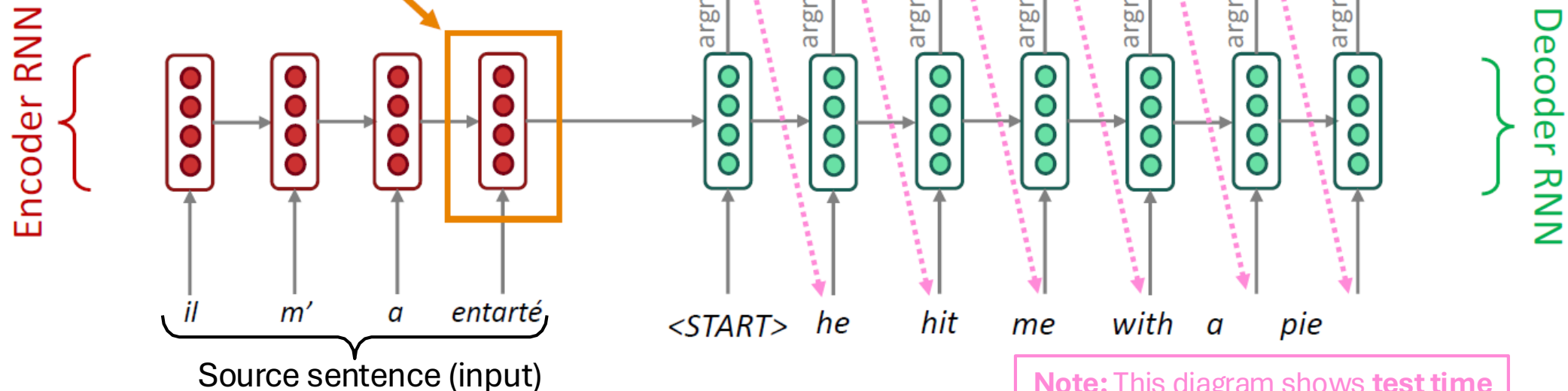
Encoder RNN produces an encoding of the source sentence.



Neural Machine Translation (NMT)

The Sequence-to-Sequence Model

Encoding of the source sentence.
Provides initial hidden state
for Decoder RNN.



Encoder RNN produces an *encoding* of the source sentence.

Decoder RNN is a Language Model that generates target sentence, conditioned on *encoding*.

Note: This diagram shows **test time** behavior: decoder output is fed in as next step's input



Neural Machine Translation (NMT)

- The **sequence-to-sequence model** is an example of a **Conditional Language Model**
 - **Language Model** because the decoder is predicting the next word of the target sentence y
 - **Conditional** because its predictions are also conditioned on the source sentence x
- NMT directly calculates $P(y|x)$

$$P(y|x) = P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots \underbrace{P(y_T|y_1, \dots, y_{T-1}, x)}$$

Probability of next target word, given
target words so far and source sentence x

- How to train an NMT system?

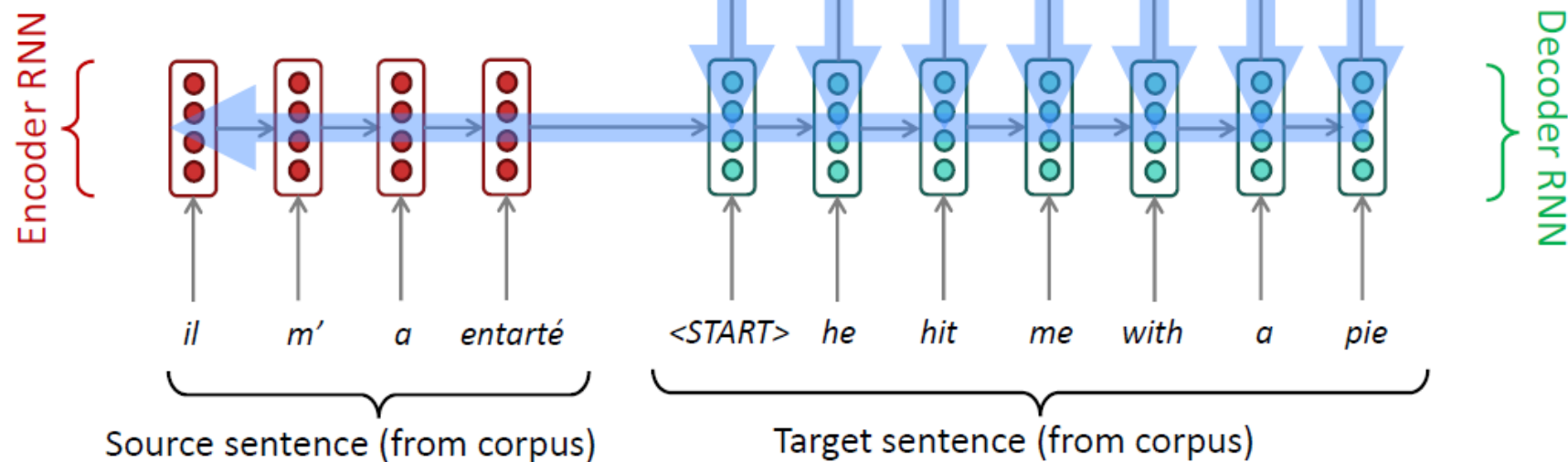


Training an NMT System

Seq2seq is optimized as a **single system**. Backpropagation operates “*end-to-end*”.

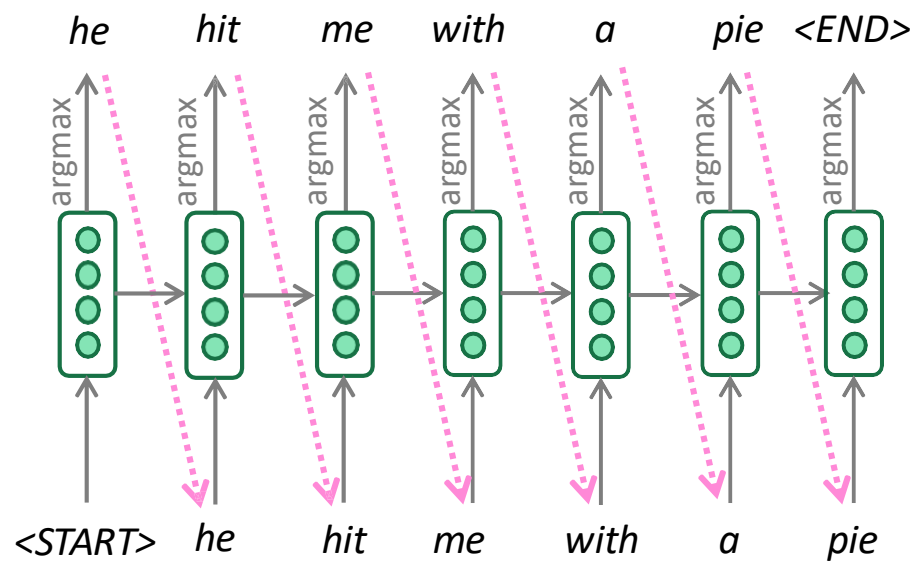
$$J = \frac{1}{T} \sum_{t=1}^T J_t = \boxed{J_1} + J_2 + J_3 + \boxed{J_4} + J_5 + J_6 + \boxed{J_7}$$

= negative log prob of “he” = negative log prob of “with” = negative log prob of <END>



Greedy decoding

- We saw how to generate (or “decode”) the target sentence by taking argmax on each step of the decoder.



- This is **greedy decoding** (take most probable word on each step)
- **Problems with this method?**



Problems With Greedy Decoding

- Greedy decoding has no way to undo decisions!
- Input: *il a m'entarté* (he hit me with a pie)
 - → *he* _____
 - → *he hit* _____
 - → *he hit a* _____ (whoops! no going back now...)

How to fix this?



Exhaustive Search Decoding

- Ideally we want to find a (length T) translation y that maximizes

$$\begin{aligned} P(y|x) &= P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots, P(y_T|y_1, \dots, y_{T-1}, x) \\ &= \prod_{t=1}^T P(y_t|y_1, \dots, y_{t-1}, x) \end{aligned}$$

- We could try computing **all possible sequences** y
- This means that on each step t of the decoder, we're tracking V^t possible partial translations, where V is vocab size
- This $O(V^T)$ complexity is **far too expensive!**



Beam Search Decoding

- **Core idea:** On each step of decoder, keep track of the ***k most probable*** partial translations (which we call ***hypotheses***)
 - k is the **beam size** (in practice around 5 to 10)
- A hypothesis $y_1, \dots, y_t$ has a **score** which is its **log probability**:

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

- Scores are all negative, and higher score is better
 - We search for high-scoring hypotheses, tracking top k on each step
- Beam search is **not guaranteed** to find optimal solution
 - But **much more efficient** than exhaustive search!



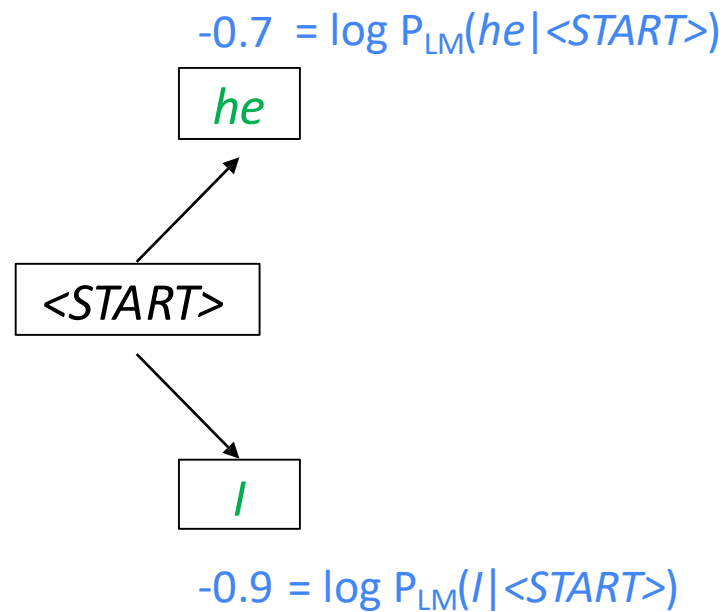
Beam Search Decoding: Example

Beam size = k = 2.

<START>

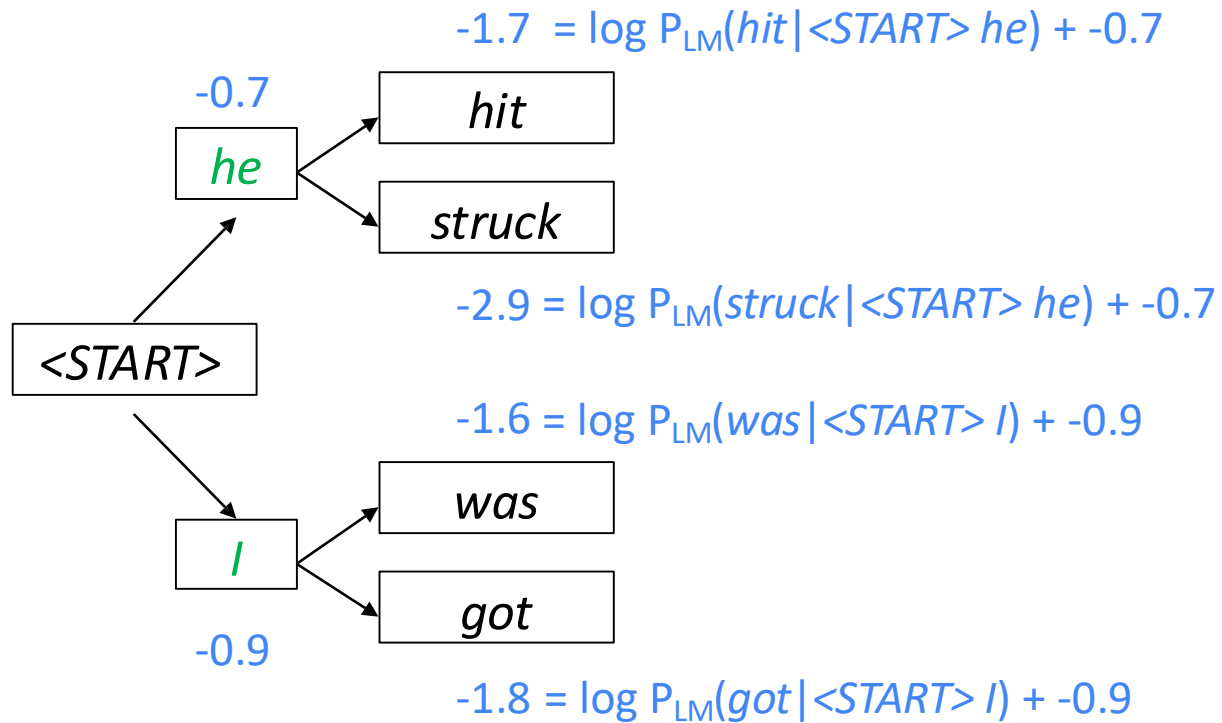
Calculate prob
distribution of next word

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



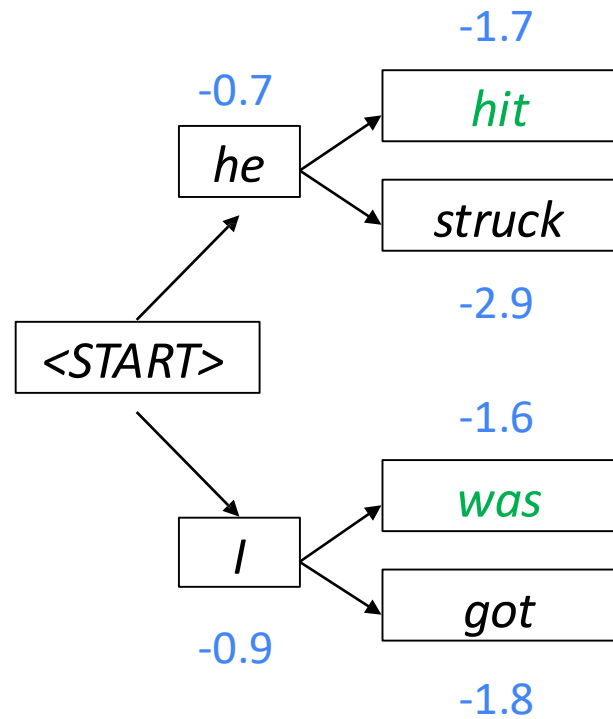
Take top k words
and compute scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



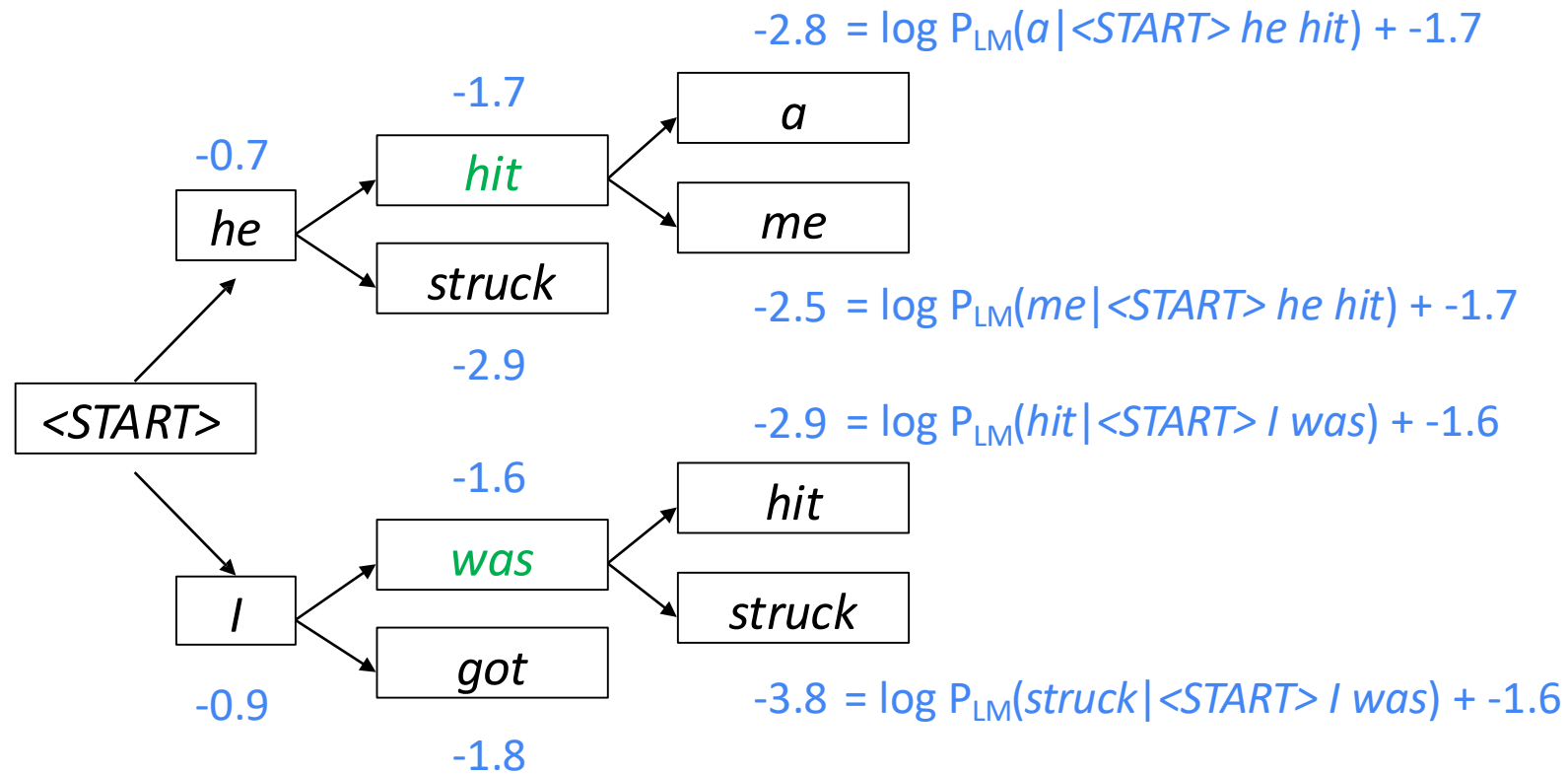
For each of the k hypotheses, find top k next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



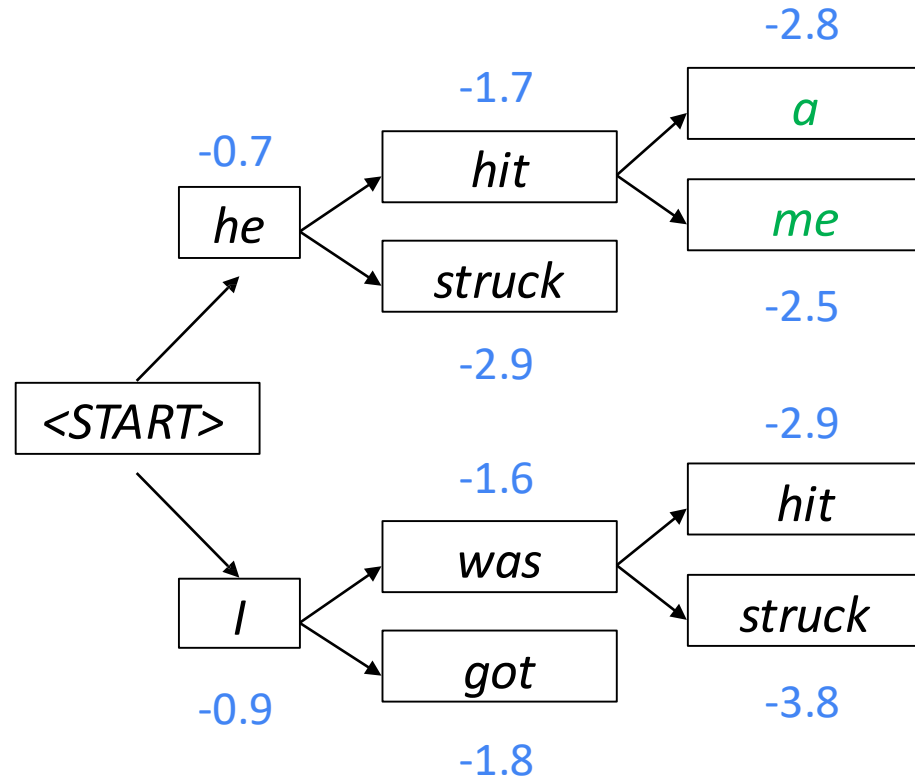
Of these k^2 hypotheses,
just keep k with highest scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



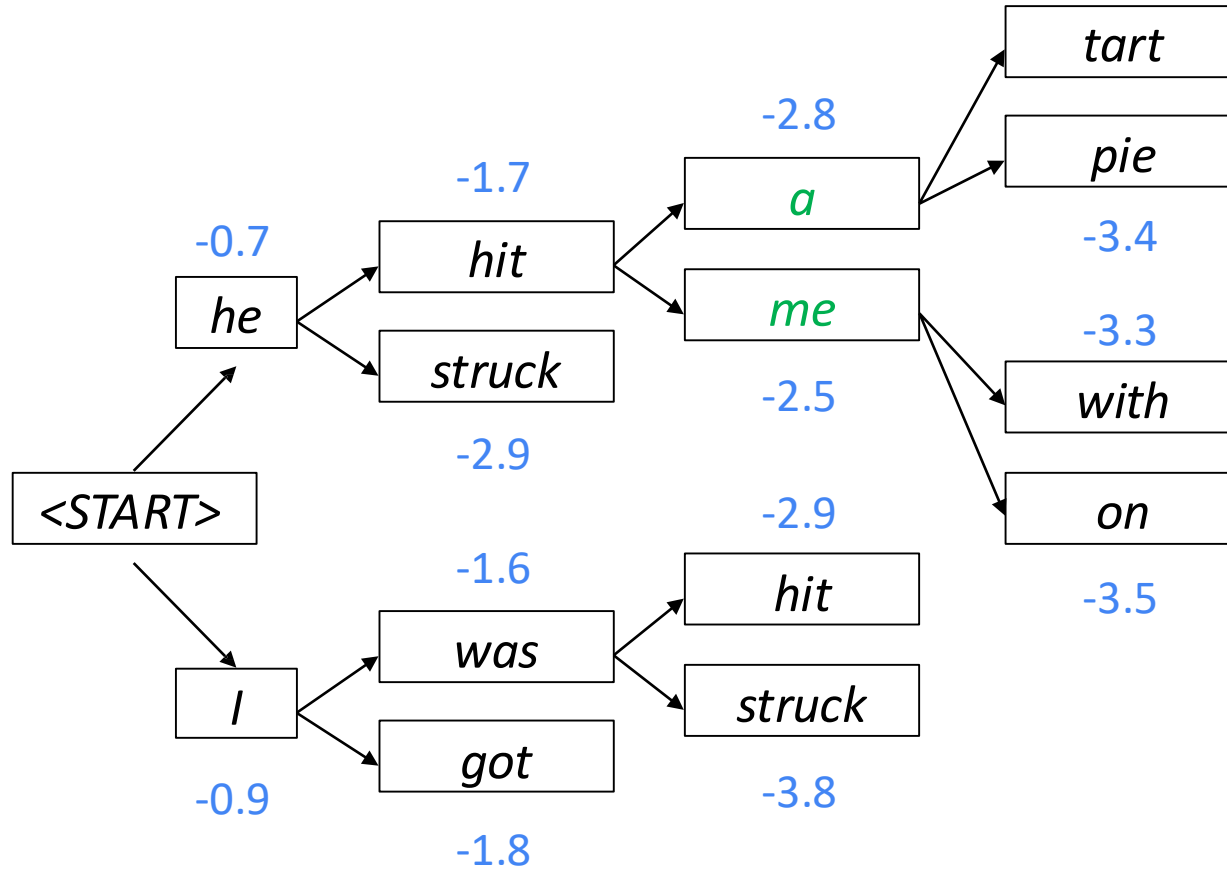
For each of the k hypotheses, find top k next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



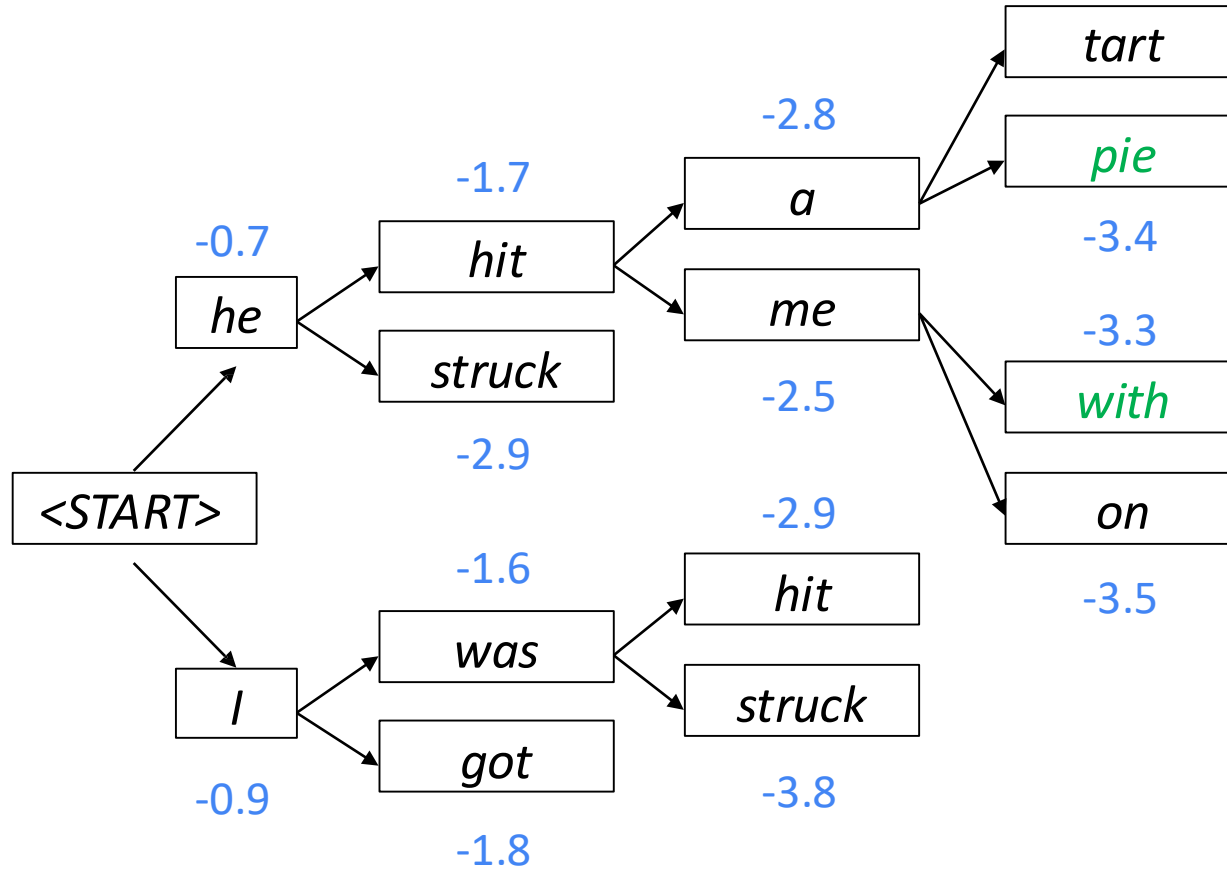
Of these k^2 hypotheses,
just keep k with highest scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



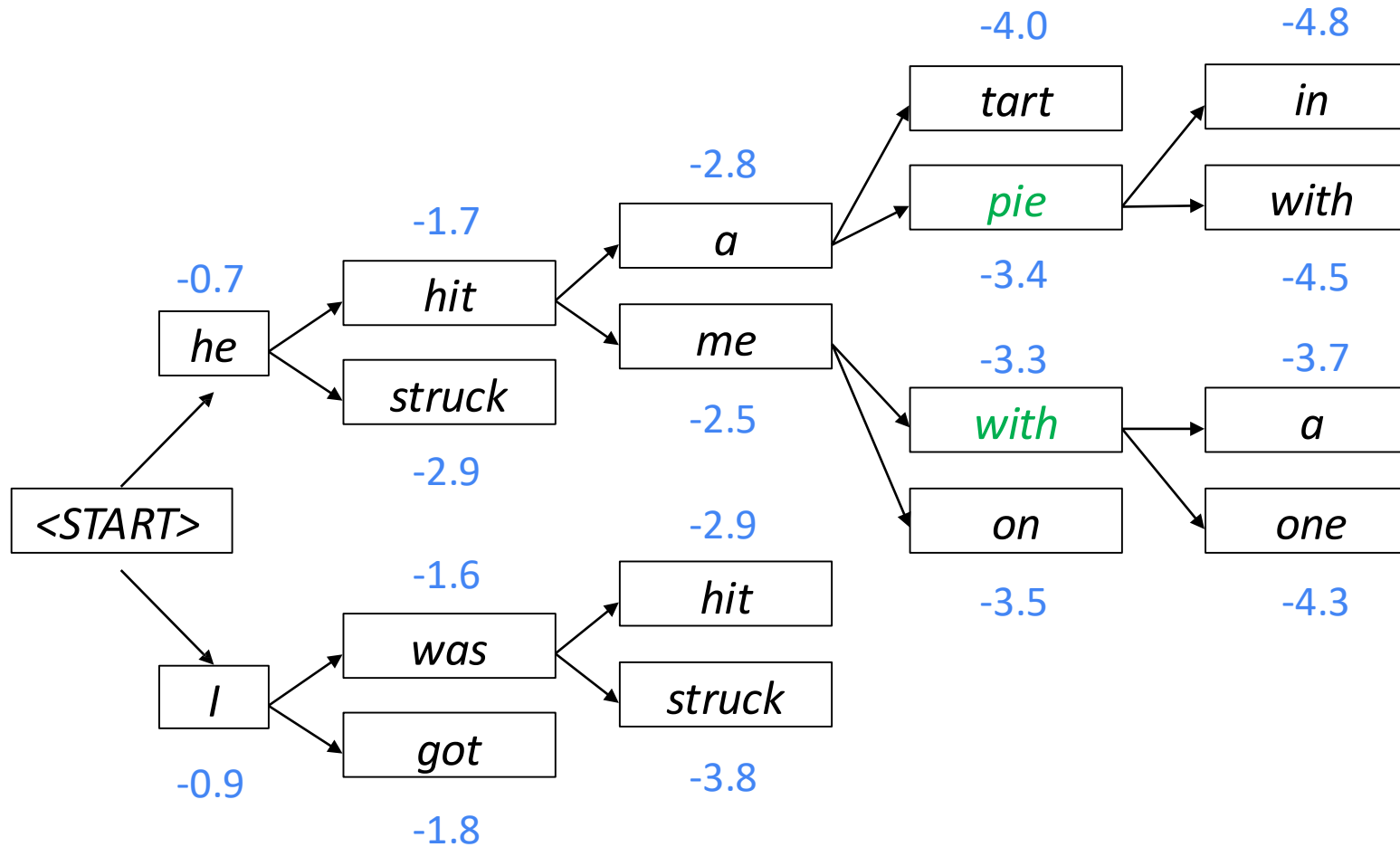
For each of the k hypotheses, find
top k next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



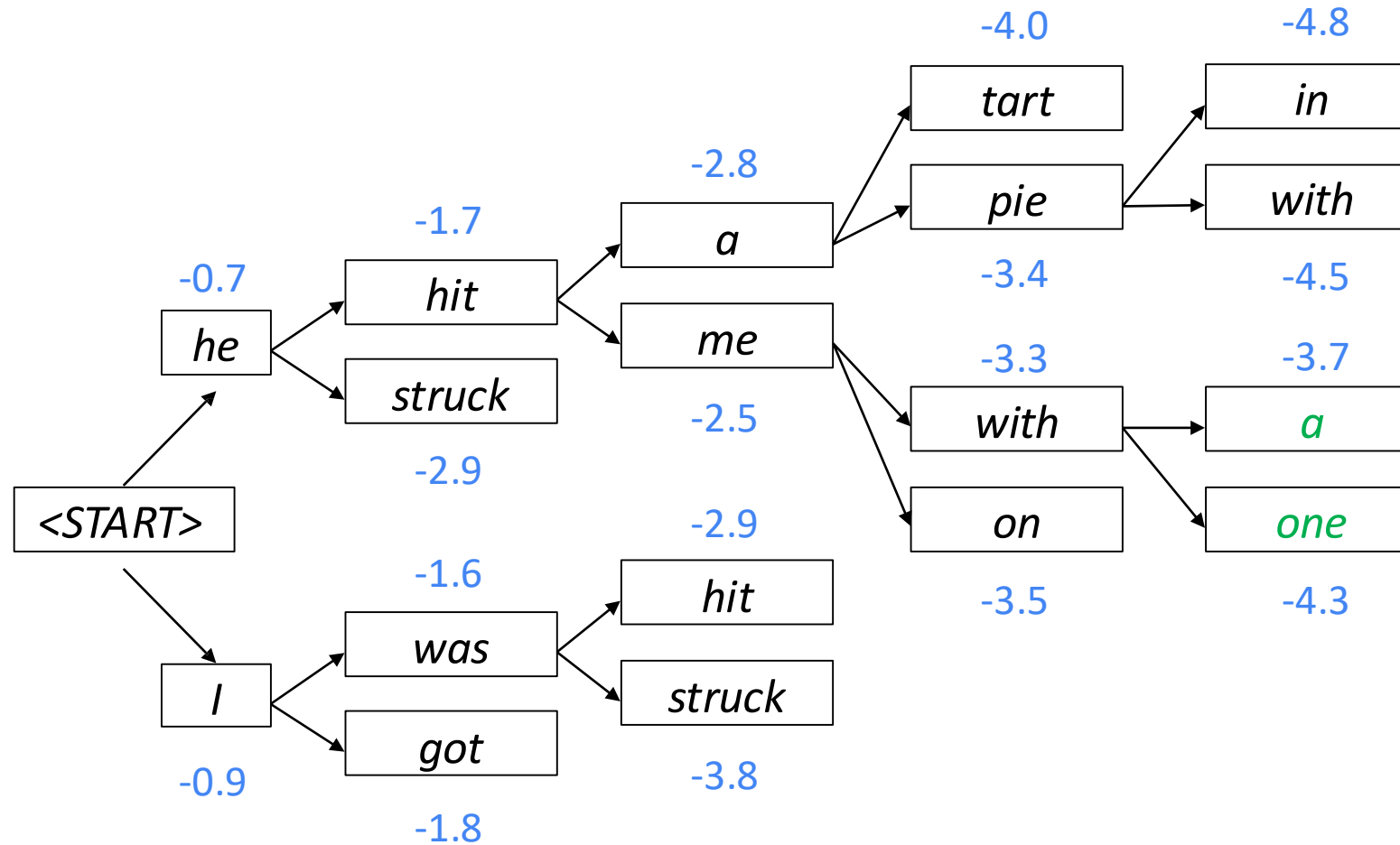
Of these k^2 hypotheses,
just keep k with highest scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



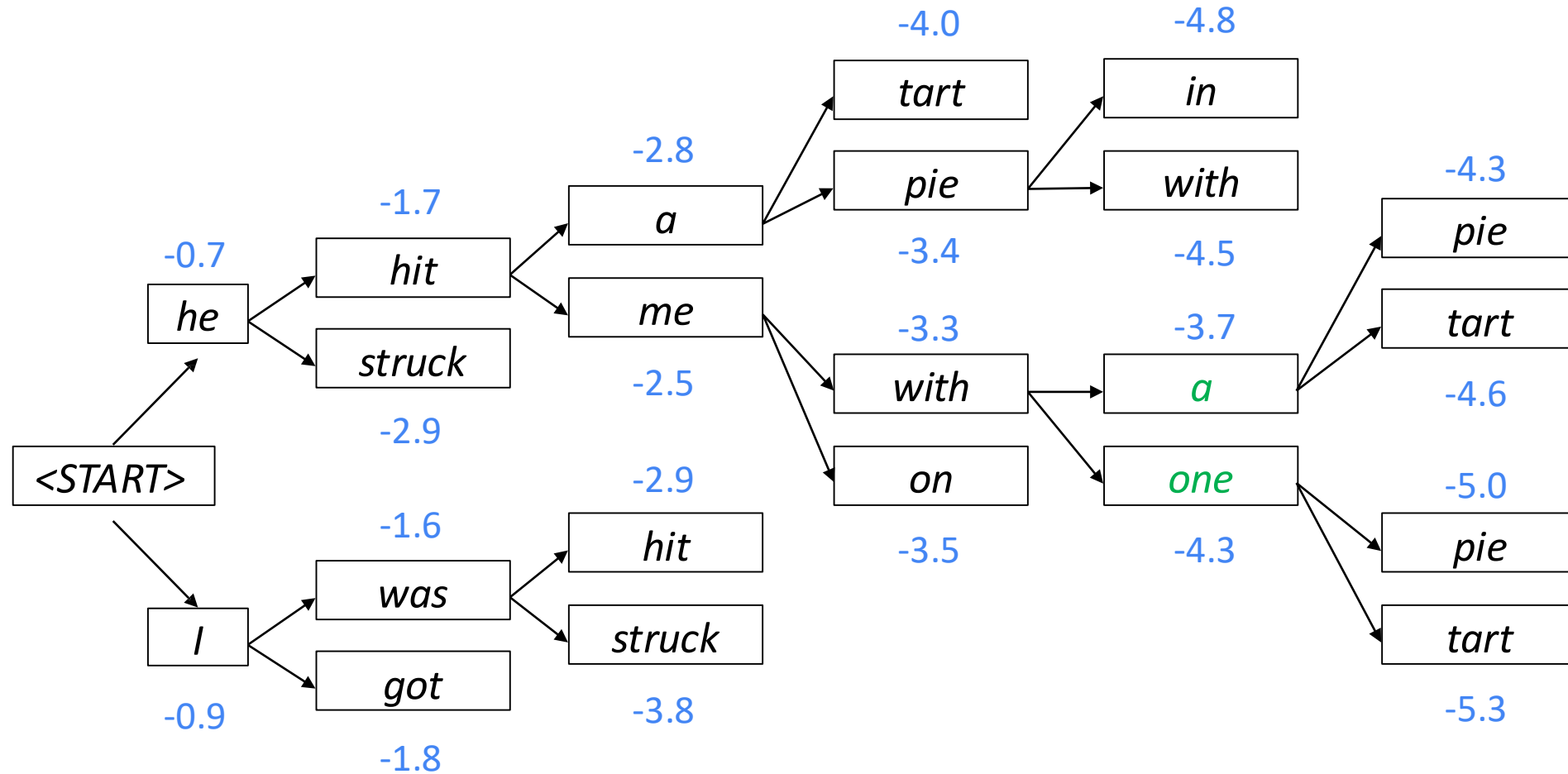
For each of the k hypotheses, find top k next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



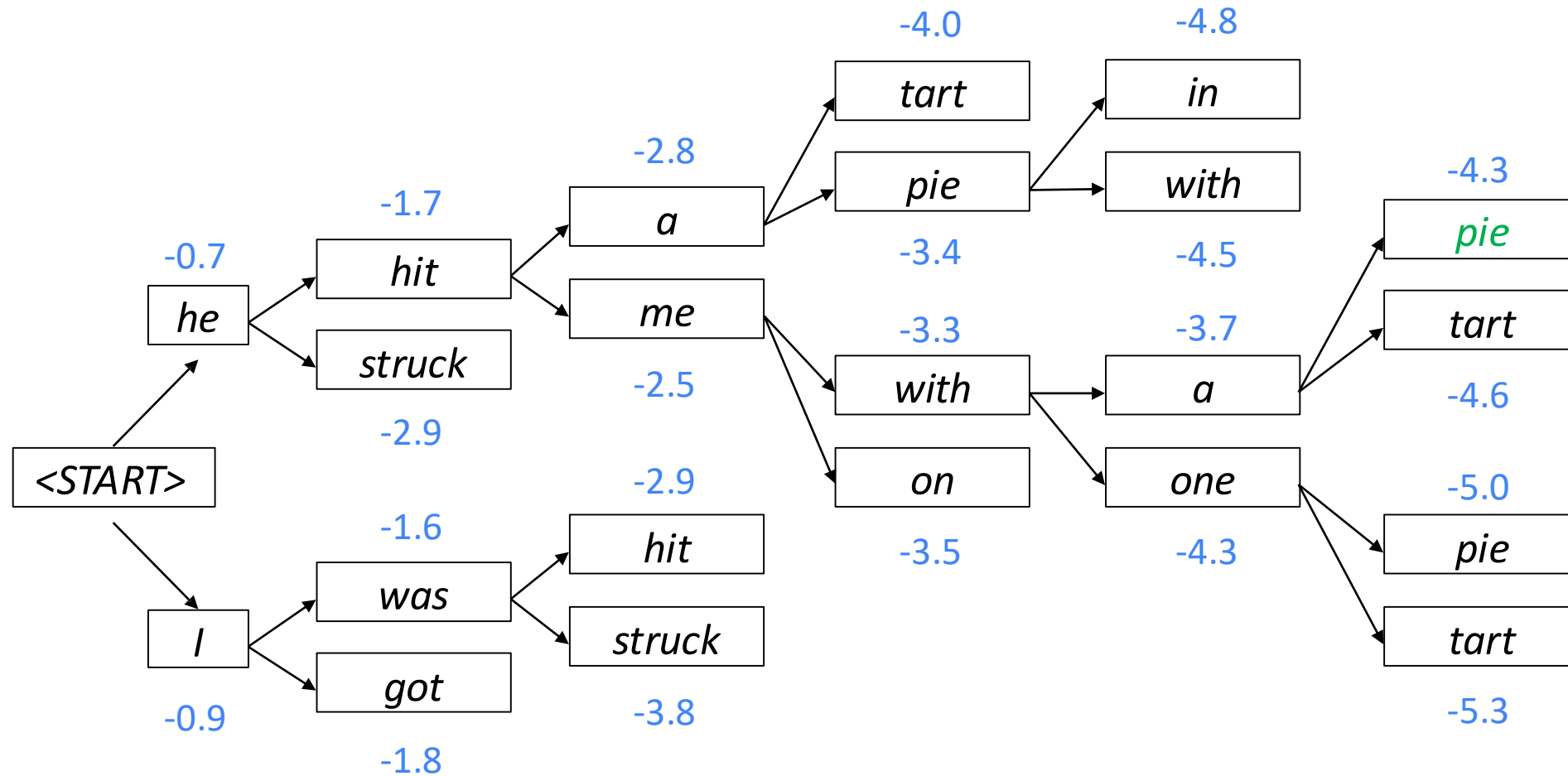
Of these k^2 hypotheses,
just keep k with highest scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



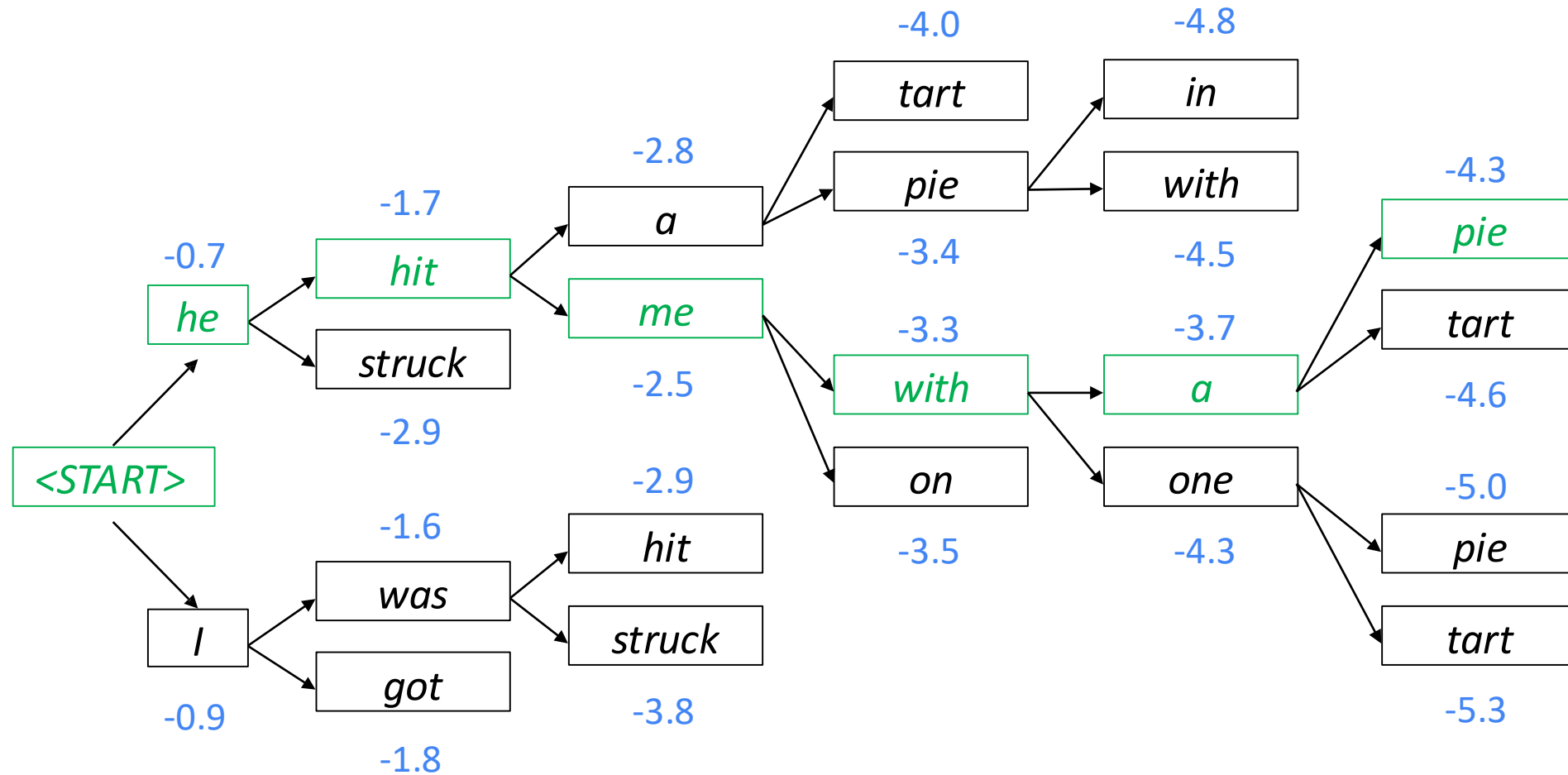
For each of the k hypotheses, find top k next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



This is the top-scoring hypothesis!

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$



Backtrack to obtain the full hypothesis

Beam Search Decoding: Stopping Criterion

- In **greedy decoding**, usually we decode until the model produces a **<END> token**
 - **For example:** <START> he hit me with a pie <END>
- In **beam search decoding**, different hypotheses may produce **<END> tokens** on **different timesteps**
 - When a hypothesis produces <END>, that hypothesis is **complete**.
 - **Place it aside** and continue exploring other hypotheses via beam search.
- Usually we continue beam search until:
 - We reach timestep T (where T is some pre-defined cutoff), or
 - We have at least n completed hypotheses (where n is pre-defined cutoff)



Beam Search Decoding: Finishing Up

- We have our list of completed hypotheses.
- How to select top one with highest score?
- Each hypothesis $y_1, \dots, y_t$ on our list has a score

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

- **Problem:** longer hypotheses have lower scores
- **Fix: Normalize by length.** Use this to select the top one instead:

$$\frac{1}{t} \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$



Issues With RNN

- Linear interaction distance
- Bottleneck problem
- Lack of parallelizability

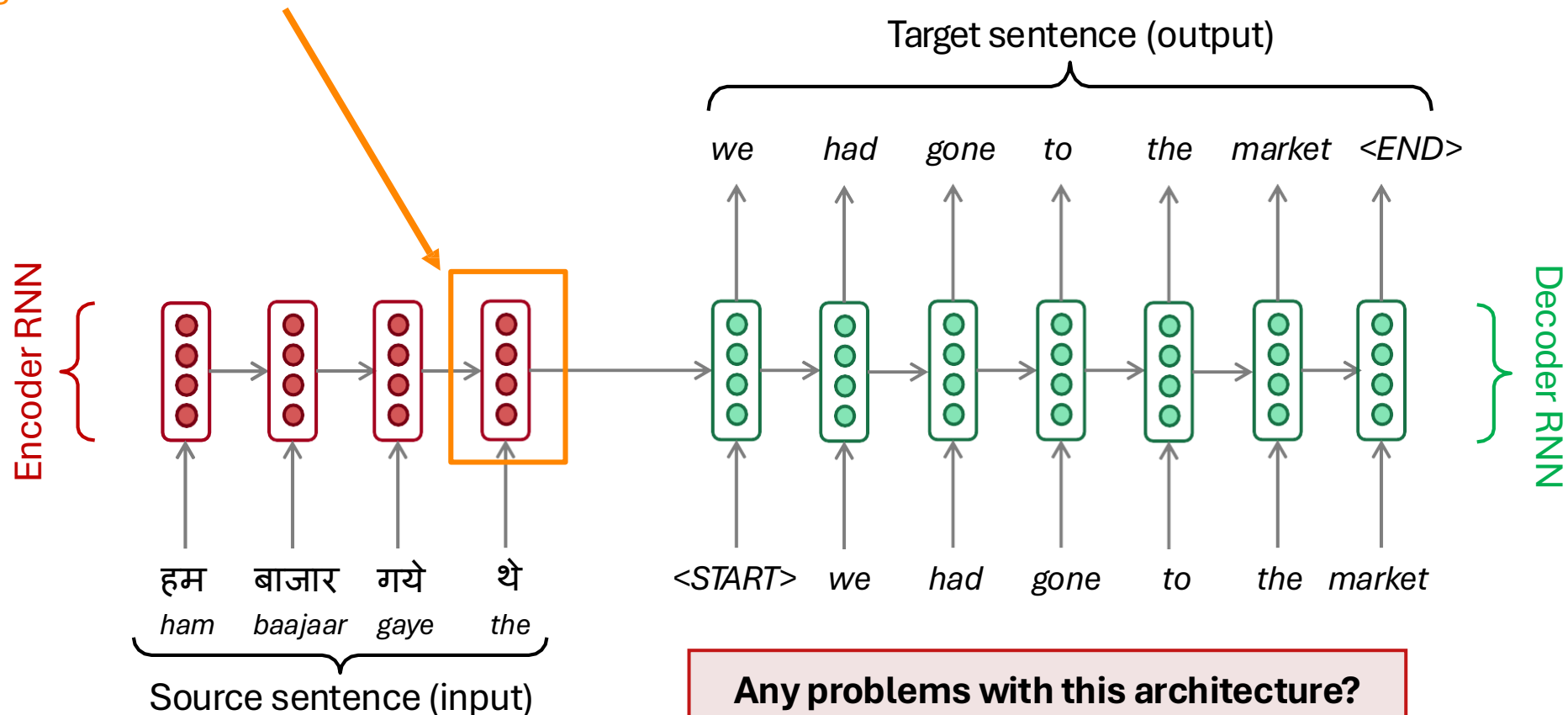
ATTENTION



Attention

Sequence-to-Sequence: The Bottleneck Problem

Encoding of the source sentence



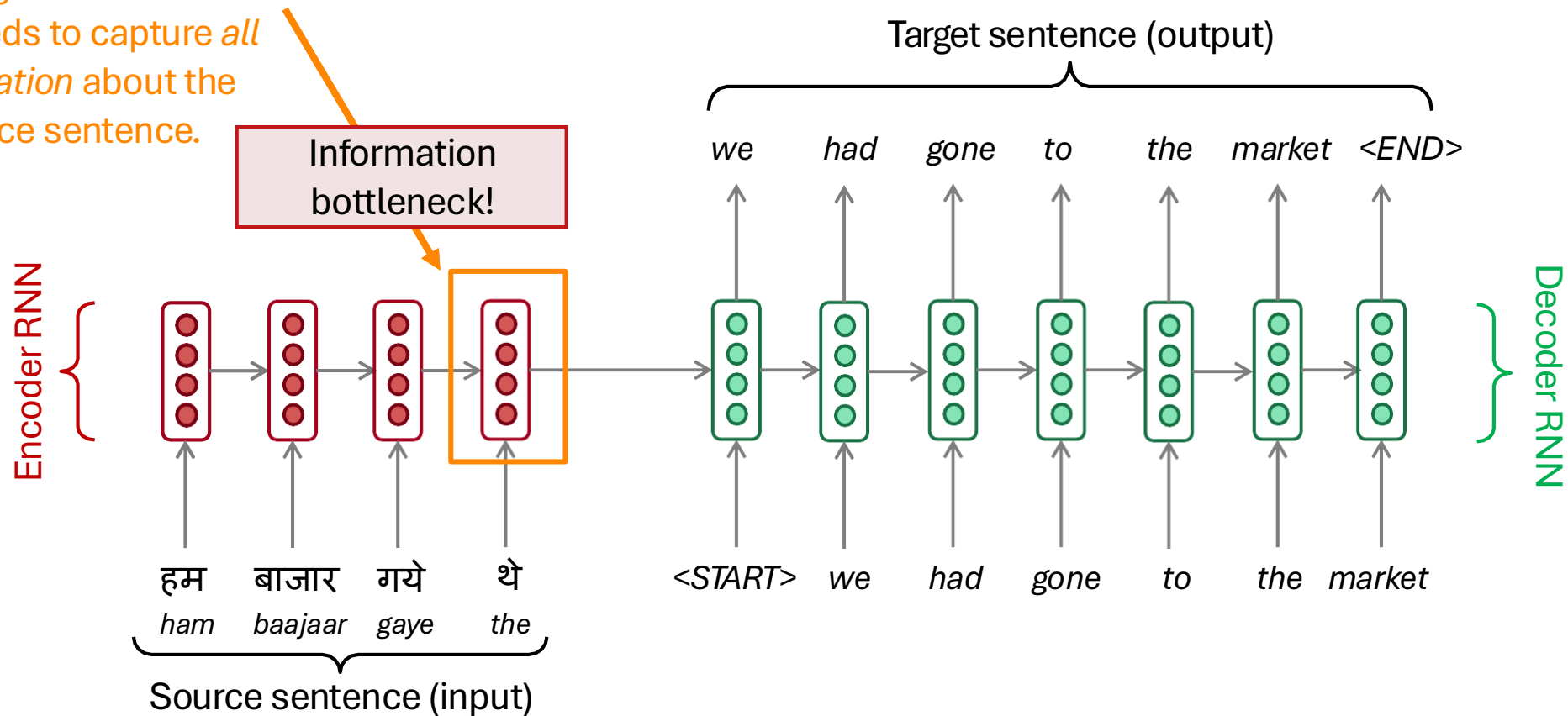
Any problems with this architecture?



Sequence-to-Sequence: The Bottleneck Problem

Encoding of the source sentence

This needs to capture *all* information about the source sentence.

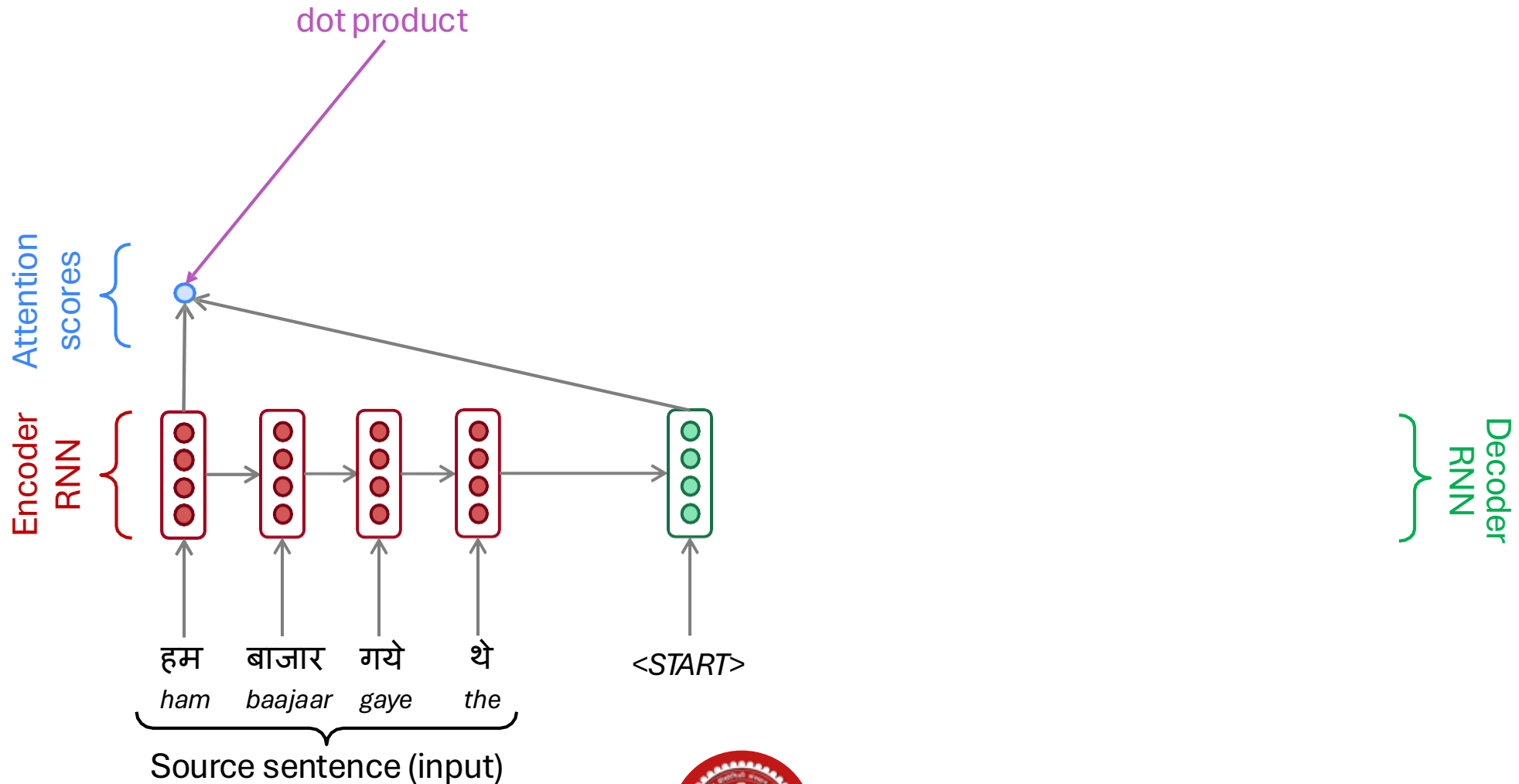


Attention

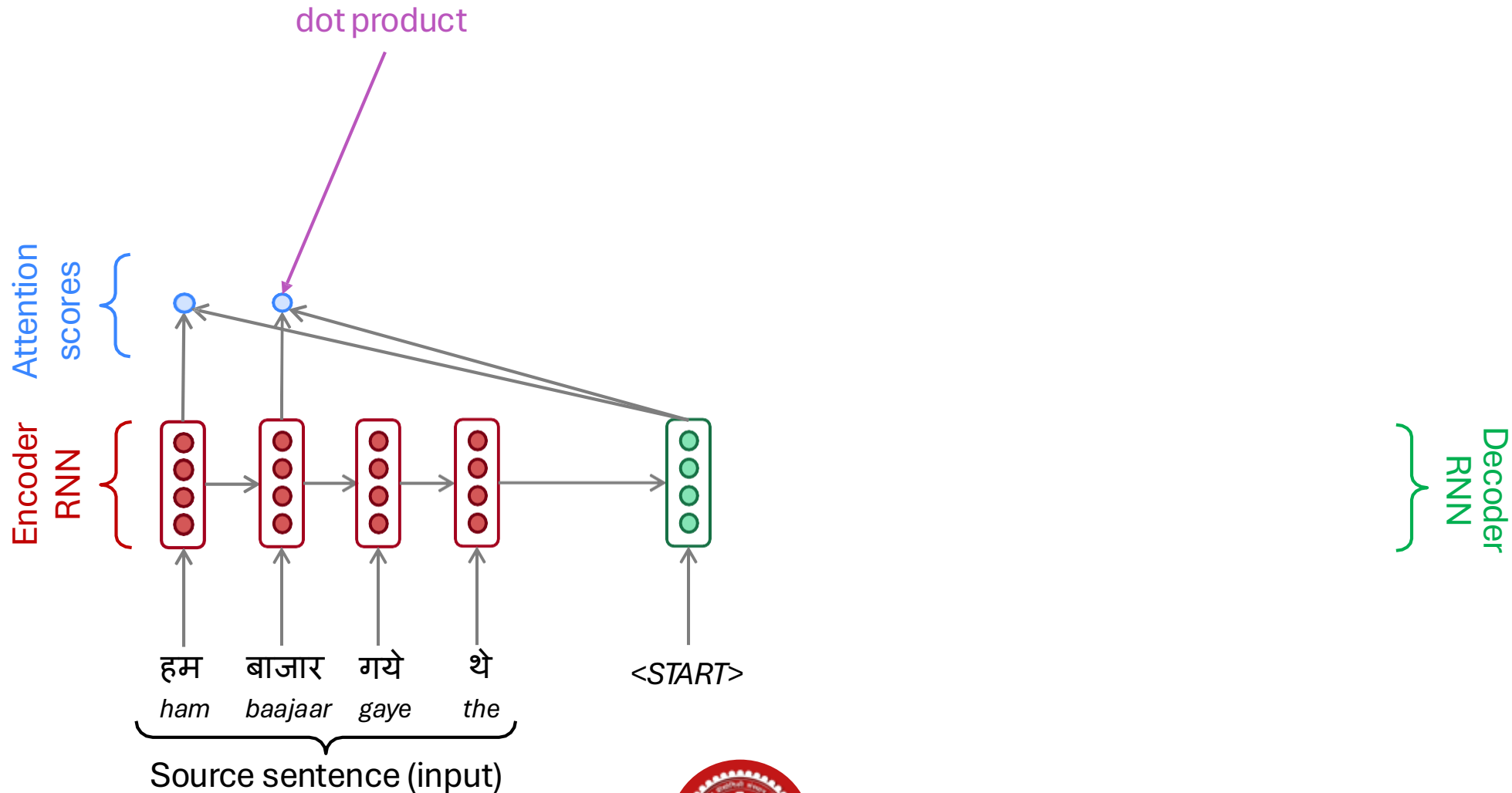
- **Attention** provides a solution to the bottleneck problem.
- **Core idea:** on each step of the decoder, use **direct connection to the encoder** to **focus on a particular part of the source sequence**
- Let's start with the visualization of the attention mechanism.



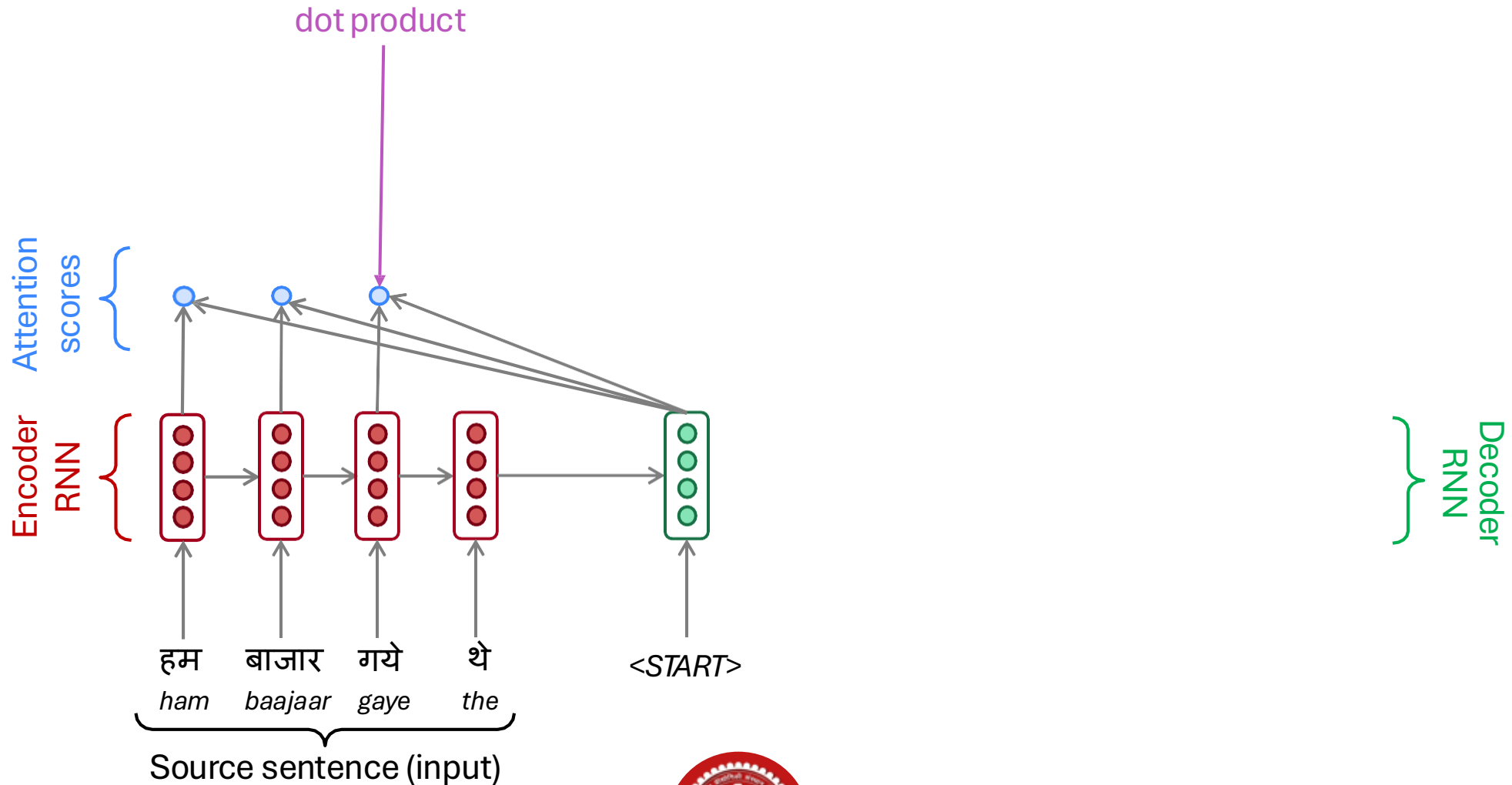
Sequence-to-Sequence With Attention



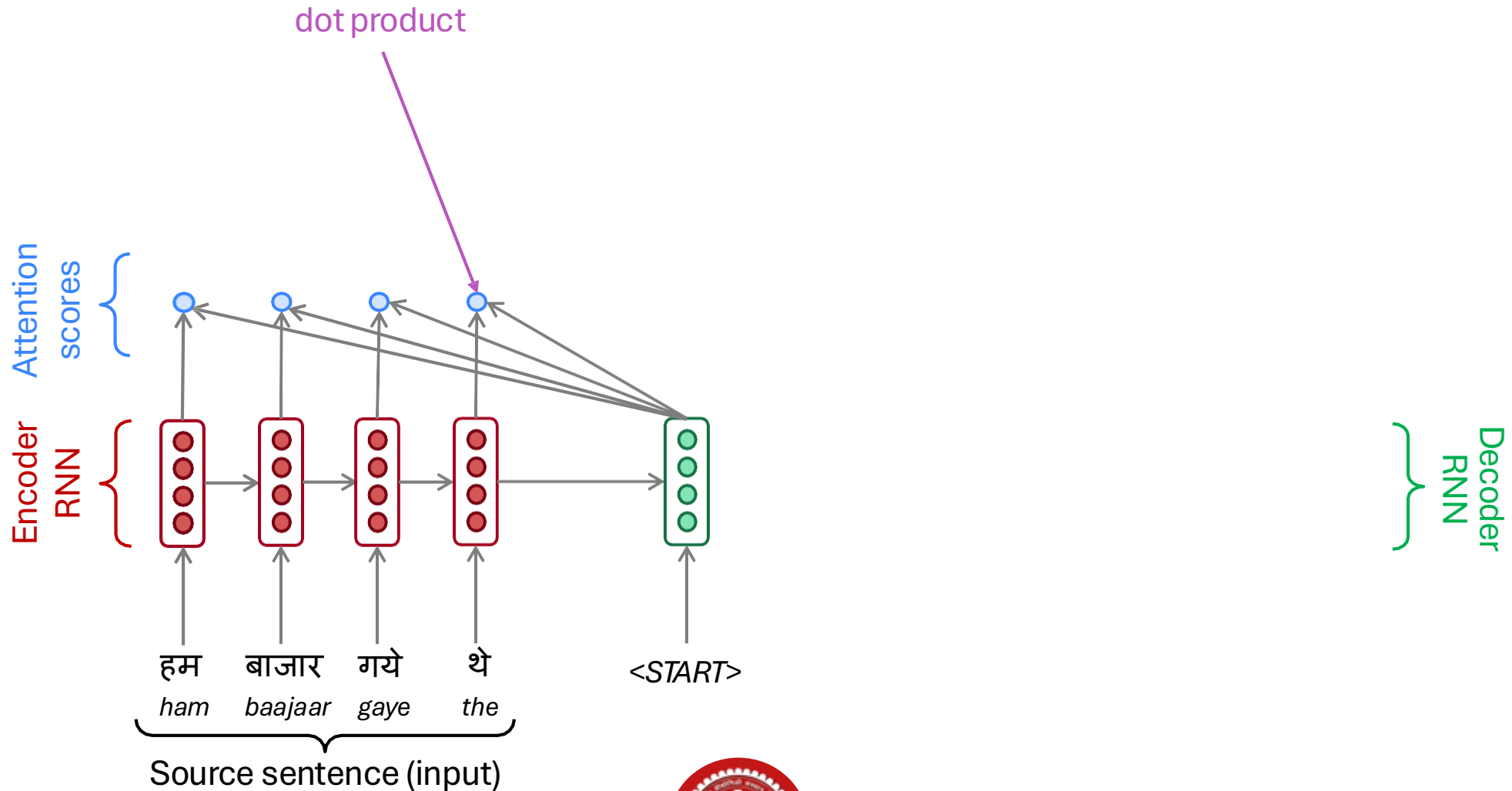
Sequence-to-Sequence With Attention



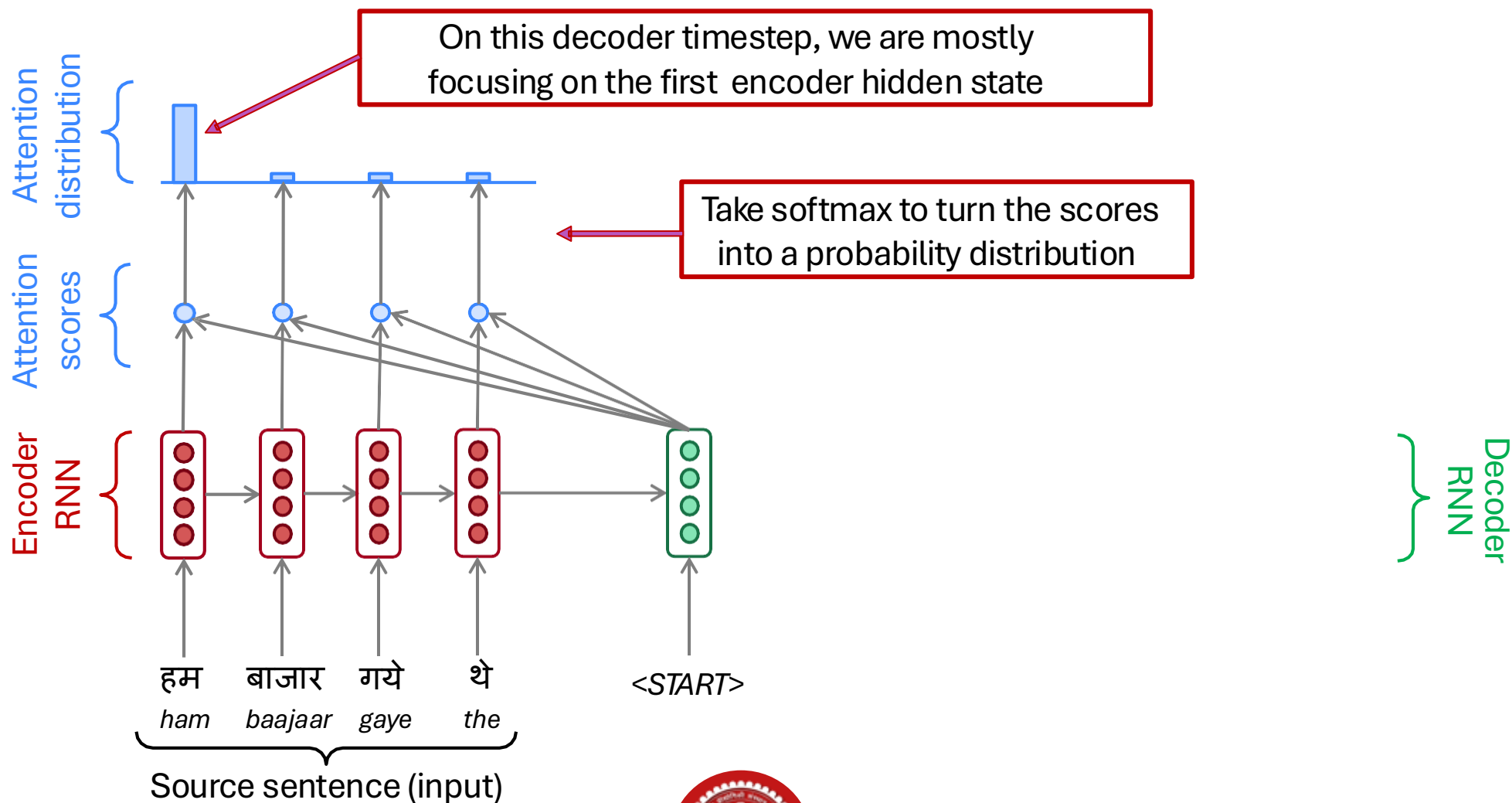
Sequence-to-Sequence With Attention



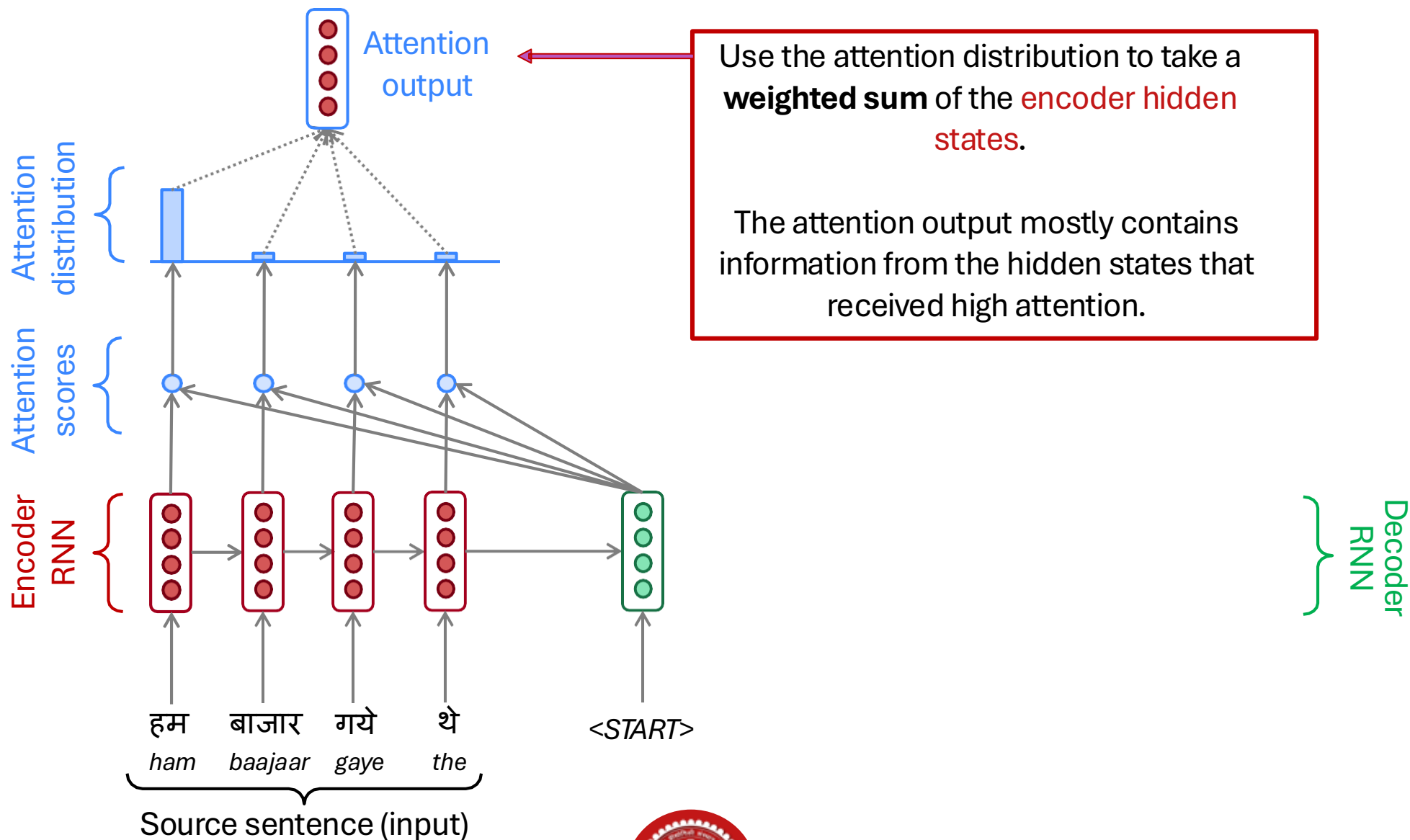
Sequence-to-Sequence With Attention



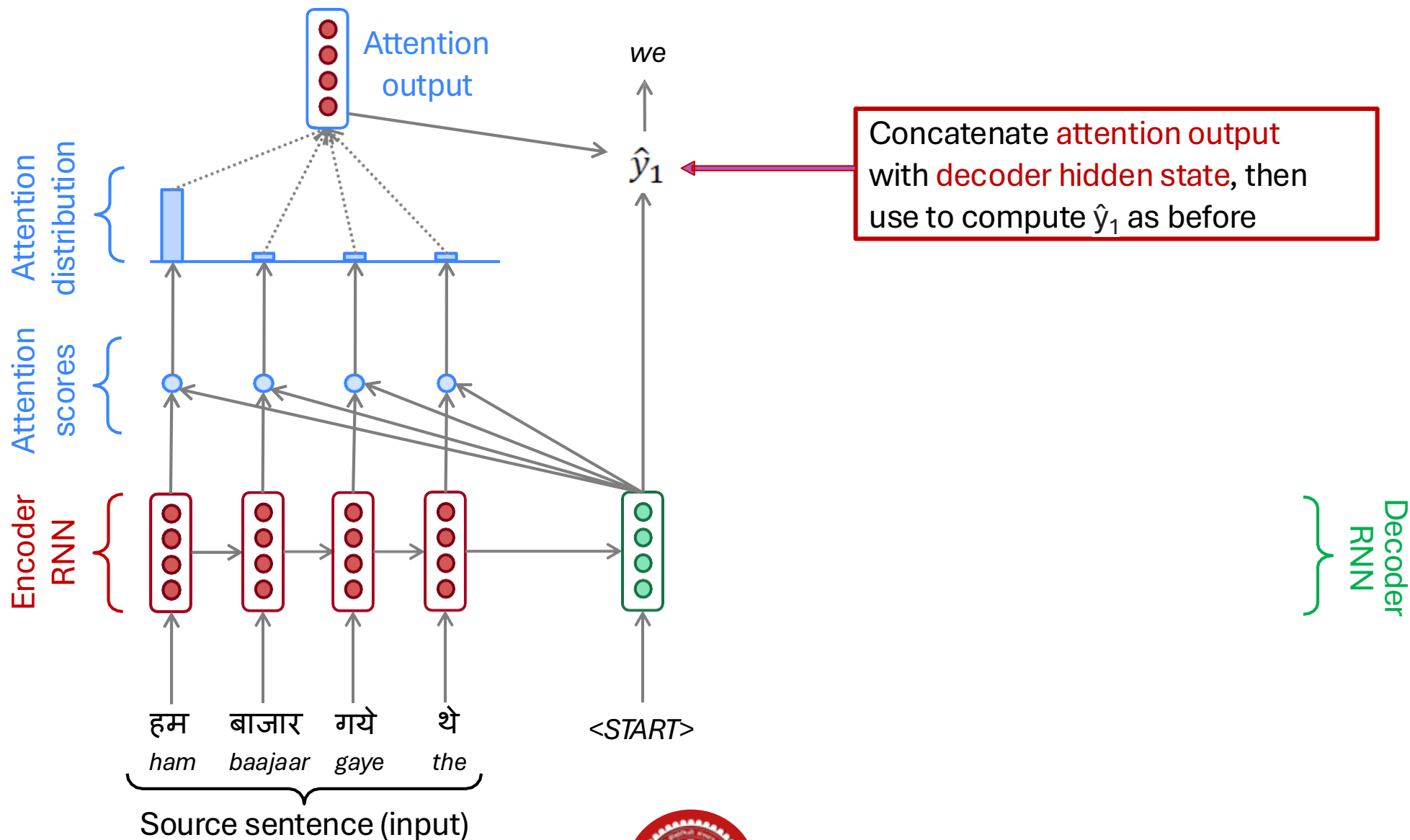
Sequence-to-Sequence With Attention



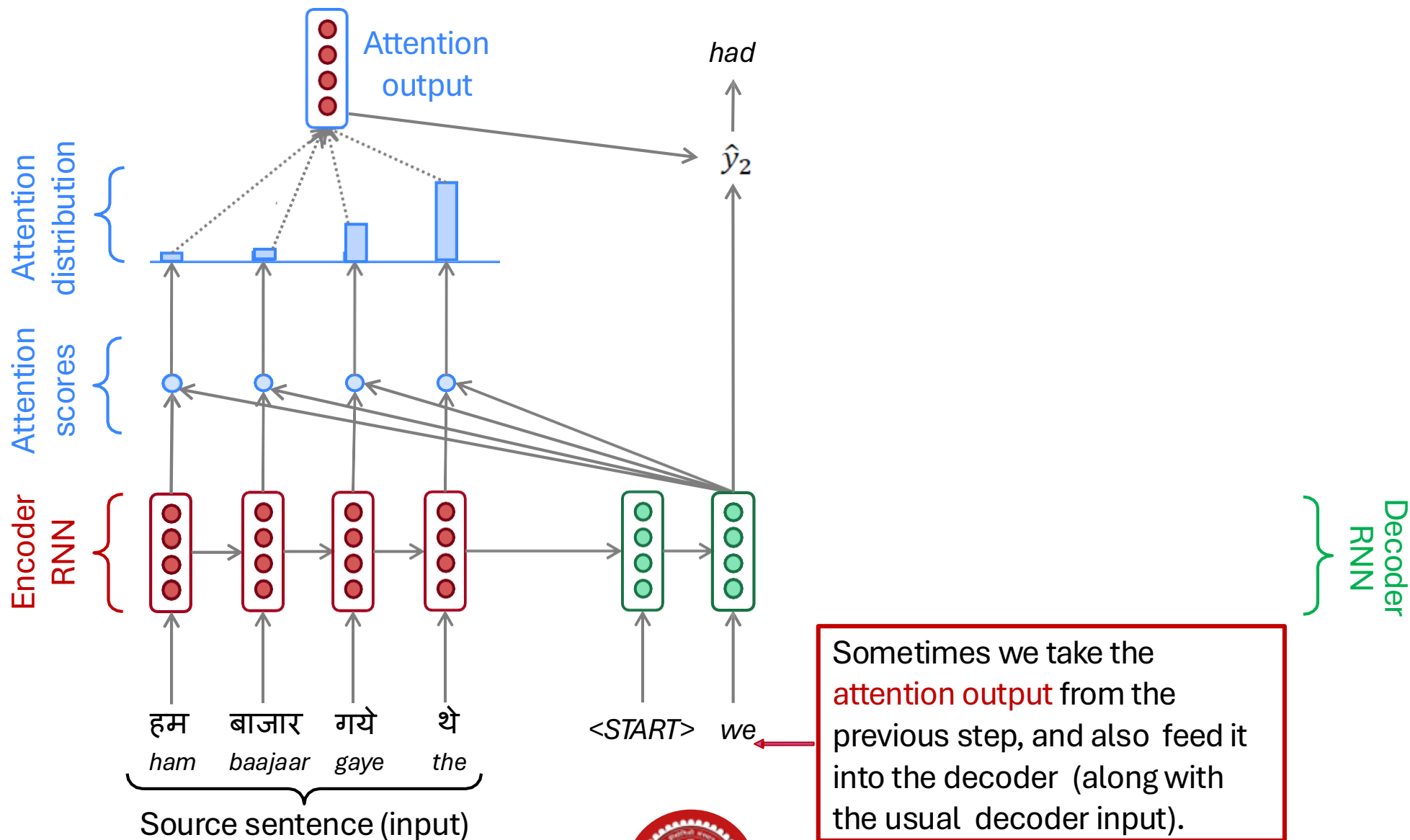
Sequence-to-Sequence With Attention



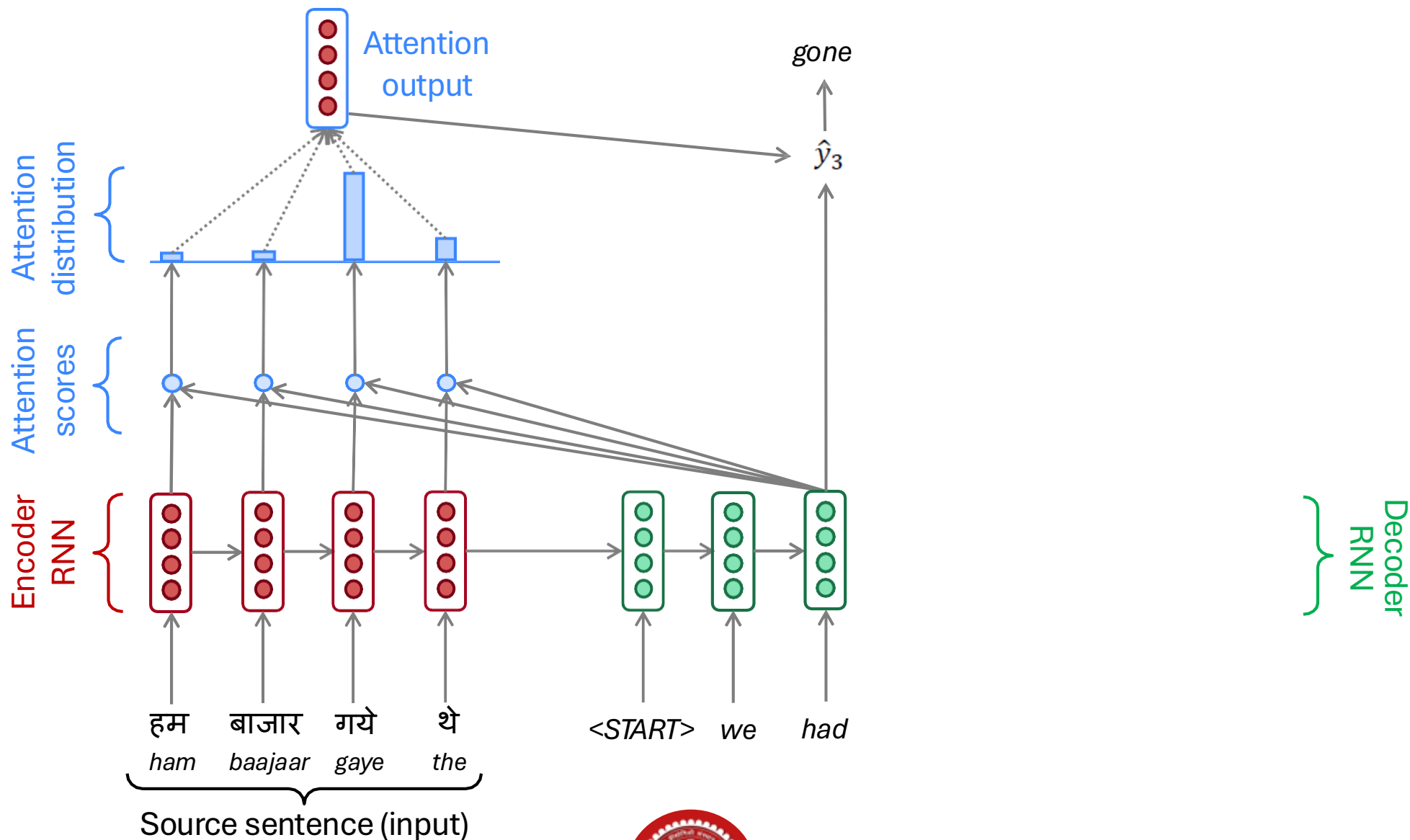
Sequence-to-Sequence With Attention



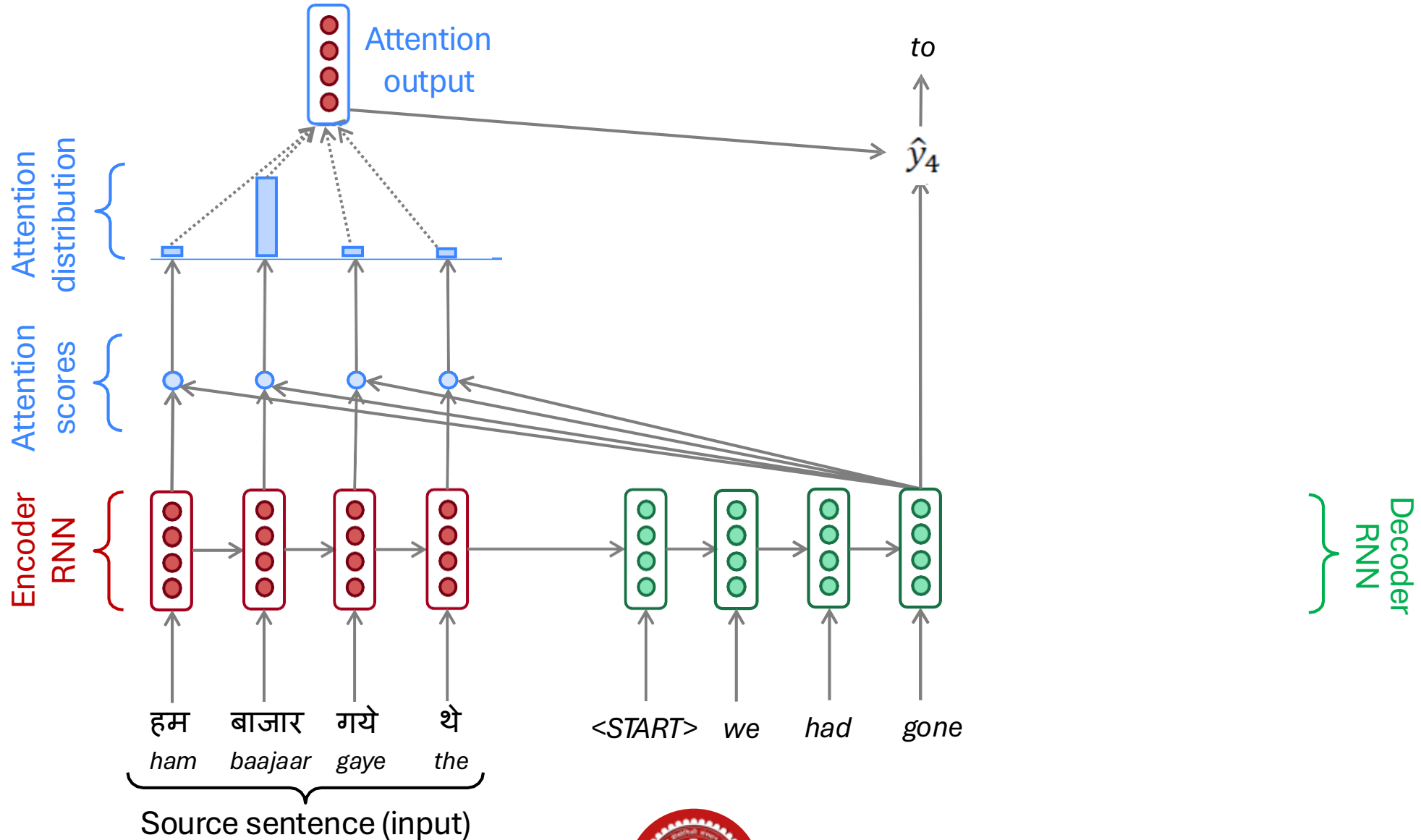
Sequence-to-Sequence With Attention



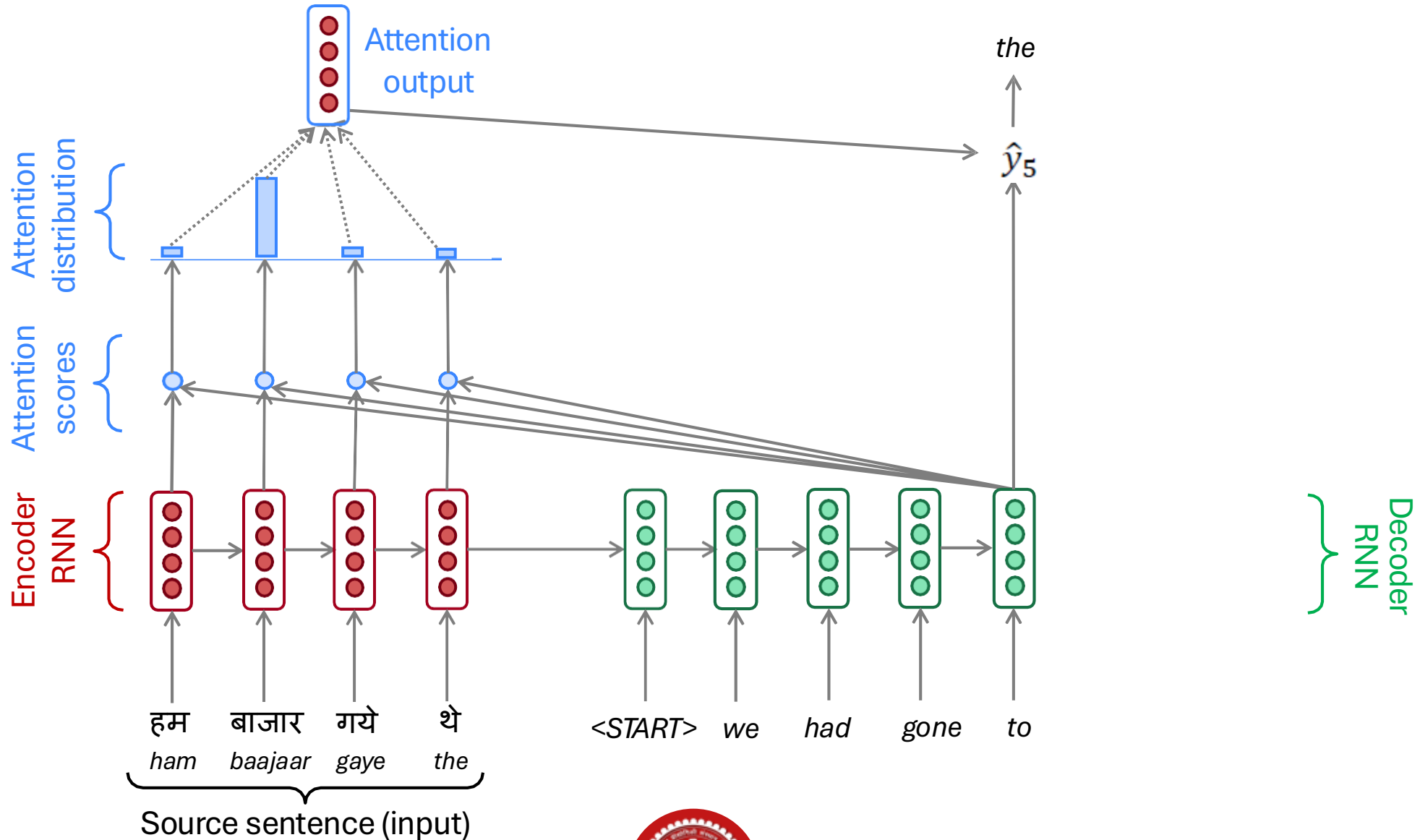
Sequence-to-Sequence With Attention



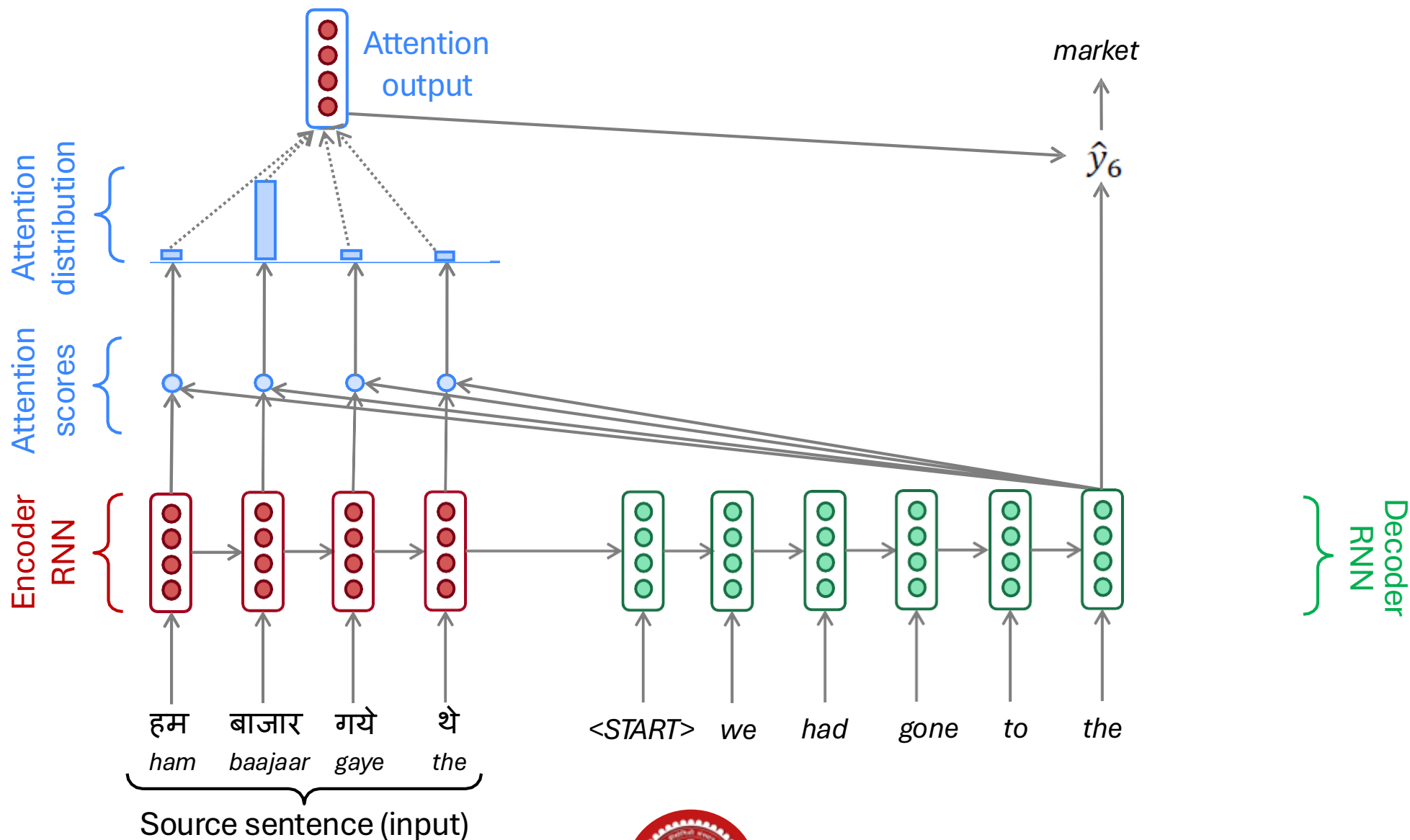
Sequence-to-Sequence With Attention



Sequence-to-Sequence With Attention



Sequence-to-Sequence With Attention



Attention: In Equations

- We have encoder hidden states $h_1, \dots, h_N \in \mathbb{R}^h$
- On timestep t , we have decoder hidden state $s_t \in \mathbb{R}^h$
- We get the attention scores e^t for this step:

$$e^t = [s_t^T h_1, \dots, s_t^T h_N] \in \mathbb{R}^N$$

- We take softmax to get the attention distribution α^t for this step (this is a probability distribution, sums to 1)

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

- We use α^t to take a weighted sum of the encoder hidden states to get the attention output a_t

$$a_t = \sum_{i=1}^N \alpha_i^t h_i \in \mathbb{R}^h$$

- Finally we concatenate the attention output a_t with the decoder hidden state s_t and proceed as in the non-attention seq2seq model

$$[a_t; s_t] \in \mathbb{R}^{2h}$$



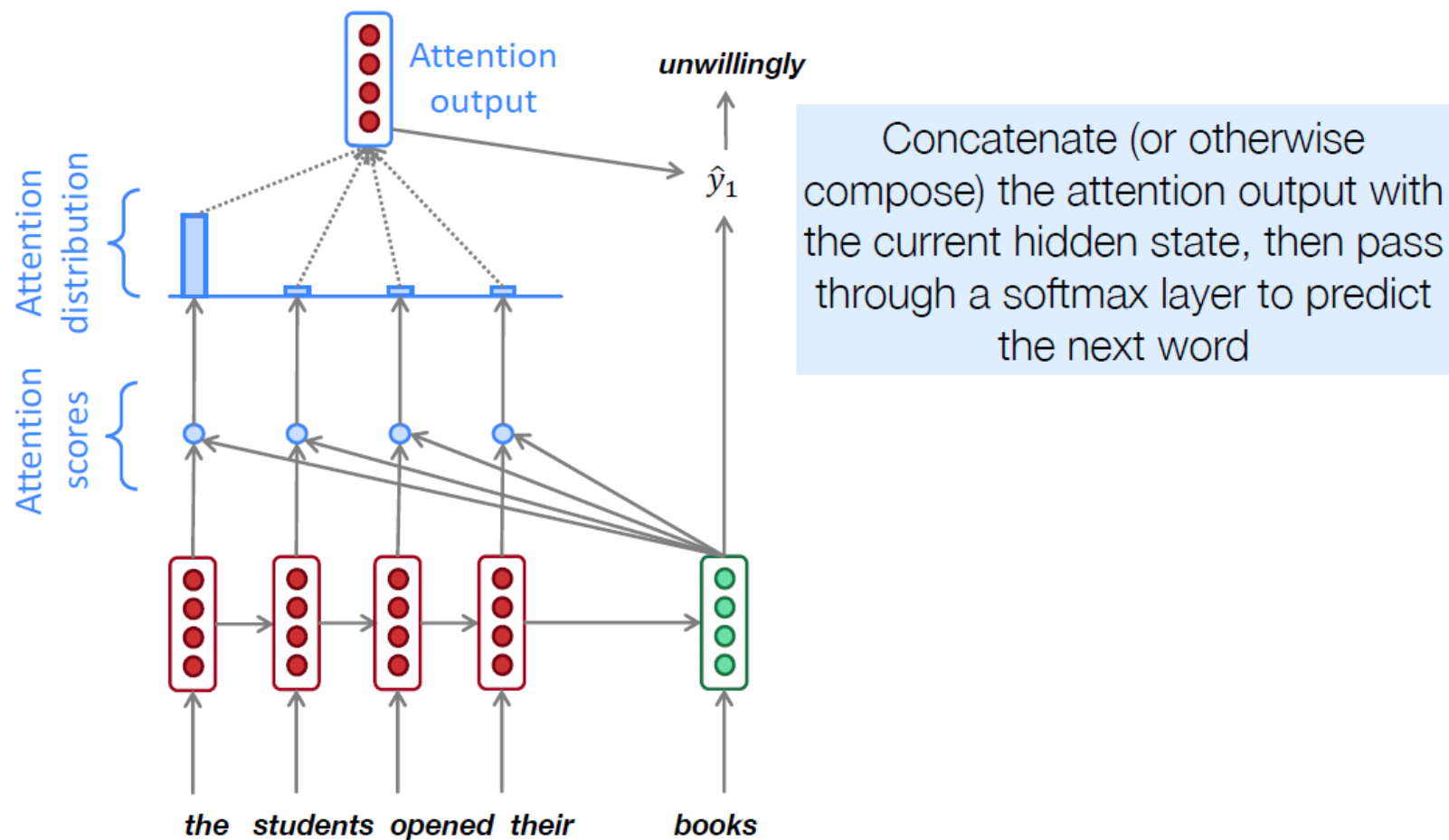
Attention is Great

- Attention significantly **improves NMT performance**
 - It's very useful to allow decoder to focus on certain parts of the source
- Attention **solves the bottleneck problem**
 - Attention allows decoder to look directly at source; bypass bottleneck
- Attention **helps with vanishing gradient problem**
 - Provides shortcut to faraway states
- Attention provides **some interpretability**
 - By inspecting attention distribution, we can see what the decoder was focusing on
 - We get (soft) **alignment for free!**
 - This is cool because we never explicitly trained an alignment system
 - The network just learned alignment by itself

	he	hit	me	with	a	pie
il						
a						
m'						
entarté						



Seq2Seq+Attention for LM



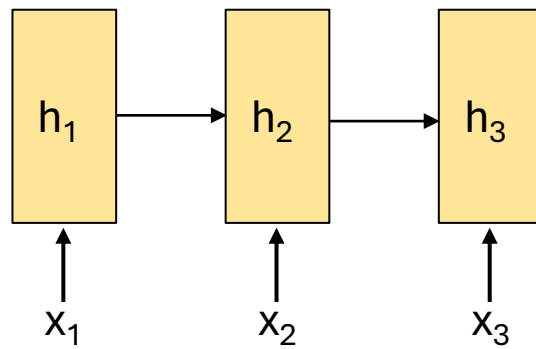
Attention is a *General* Deep Learning Technique

- We've seen that attention is a great way to improve the sequence-to-sequence model for Machine Translation.
- However: You can use attention in *many architectures* (not just seq2seq) and *many tasks* (not just MT)
- **More general definition of attention:**
 - Given a set of vector *values*, and a vector *query*, **attention** is a technique to compute a weighted sum of the values, dependent on the query.
- We sometimes say that the *query attends to the values*.
- For example, in the seq2seq + attention model, each decoder hidden state (query) *attends to* all the encoder hidden states (values).
- **Intuition:**
 - The weighted sum is a *selective summary* of the information contained in the values, where the query determines which values to focus on.
 - Attention is a way to obtain a *fixed-size representation of an arbitrary set of representations* (the values), dependent on some other representation (the query).



Attention

Encoding



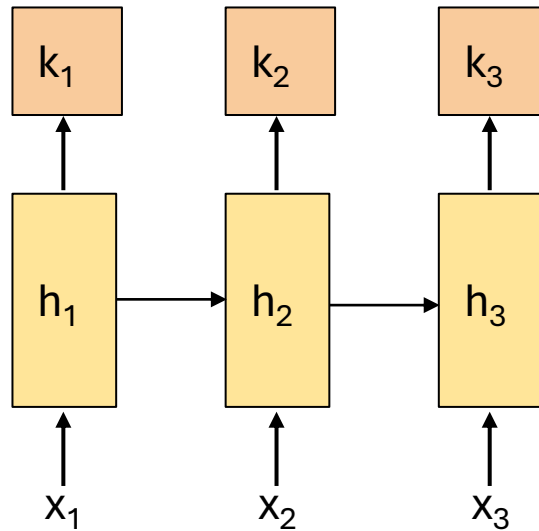
Input Sequence



Attention

Key vectors represent what **information** is **encoded** at each encoder time step.

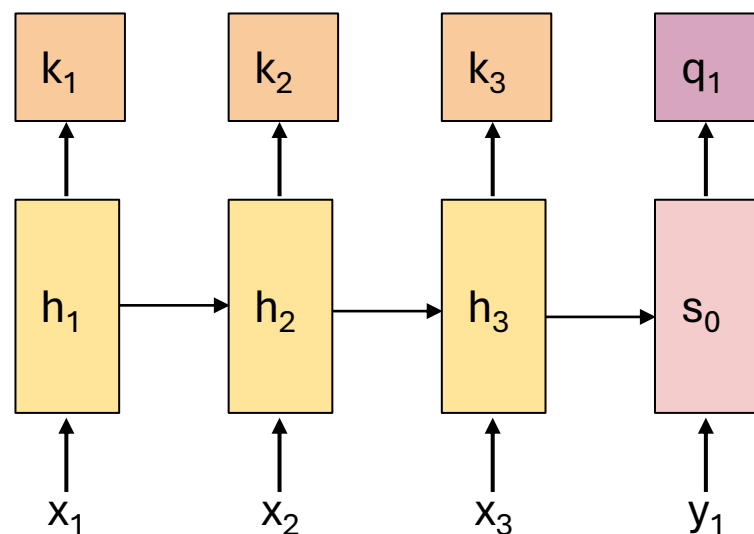
Encoding



Input Sequence



Attention



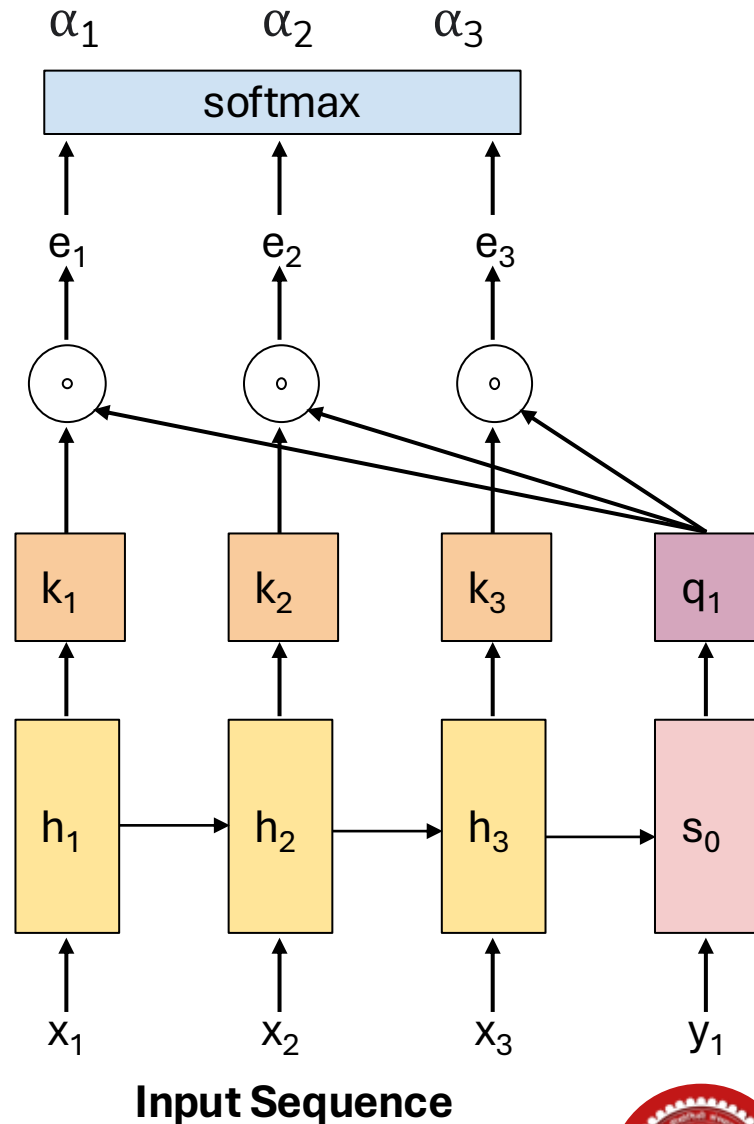
Input Sequence

Decoding

Query vectors represent what information we are **looking for** at each decoder time step.



Attention



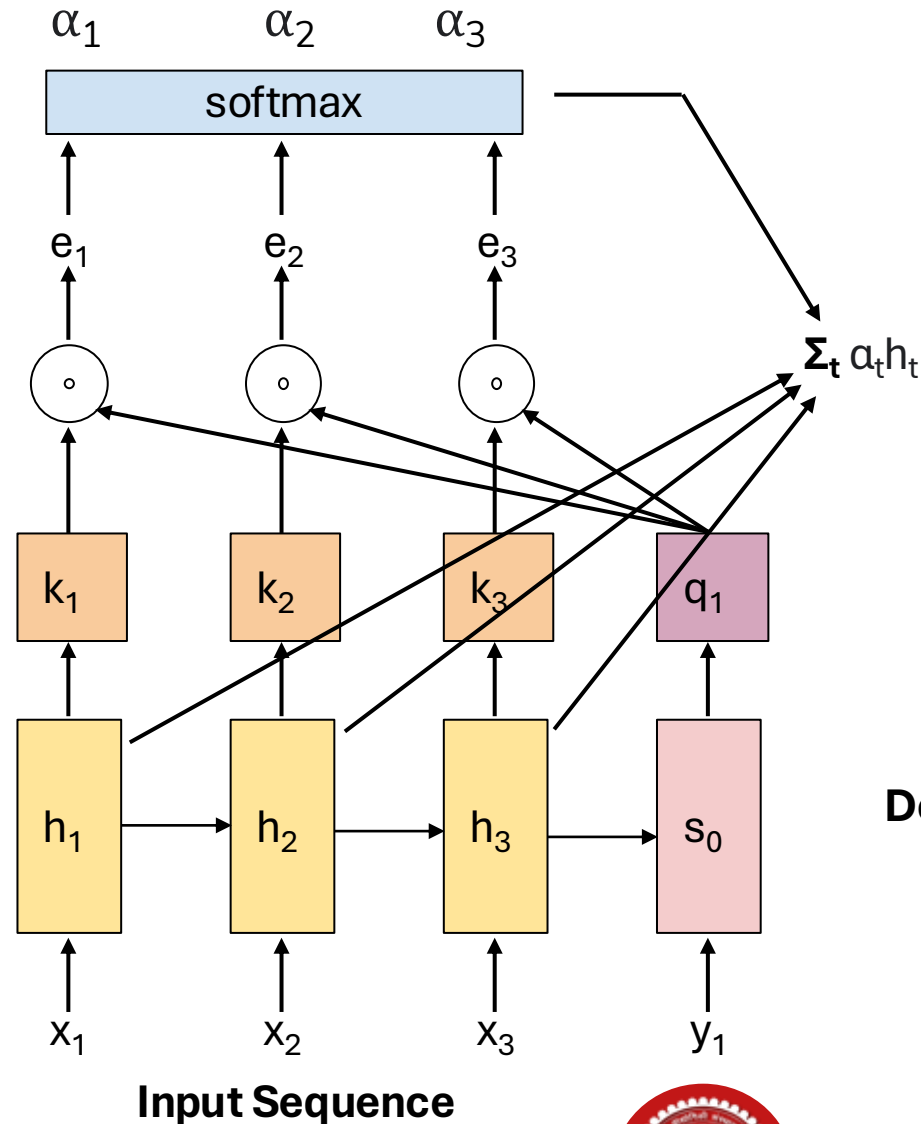
Softmax converts the similarity scores into a **probability distribution**.

Dot product between query vector and every key vector gives **similarity score**.

Decoding



Attention



The output of attention mechanism is the **weighted sum** of hidden vectors.

Instead of simply summing up the hidden vectors, we can transform them using a learned function to generate **value vectors** and then compute a weighted sum.

Decoding



Variants of Attention

- Original formulation: $a(\mathbf{q}, \mathbf{k}) = w_2^T \tanh(W_1[\mathbf{q}; \mathbf{k}])$
- Bilinear product: $a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^T W \mathbf{k}$ Luong et al., 2015
- Dot product: $a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^T \mathbf{k}$ Luong et al., 2015
- Scaled dot product: $a(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q}^T \mathbf{k}}{\sqrt{|\mathbf{k}|}}$ Vaswani et al., 2017

More information:

“Deep Learning for NLP Best Practices”, Ruder, 2017. <http://ruder.io/deep-learning-nlp-best-practices/index.html#attention>

“Massive Exploration of Neural Machine Translation Architectures”, Britz et al, 2017, <https://arxiv.org/pdf/1703.03906.pdf>

